

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DEPARTMENT OF INFORMATION ENGINEERING

COMPUTER ENGINEERING'S MASTER DEGREE

Hardening Docker Workloads on Proxmox VE: Runtime Isolation, MAC Enforcement, and Attack-Scenario Analysis

Fabio Baldan

Giancarlo Padoan

ANNO ACCADEMICO 2025-2026

Hackers don't need names or proximity - just opportunity.
-Robert Mueller

Executive Summary

This paper argues that securing Docker workloads on Proxmox VE requires layered hardening rather than reliance on container isolation alone. It combines a conceptual analysis of Linux isolation primitives and Mandatory Access Control (MAC) systems with a qualitative scenario analysis on a four-layer stack comprising the Proxmox node, guest VMs, an LSM-enforced runtime, and container workloads, as shown in Figure 6.1. Across the scenarios, the paper’s main finding is consistent: patching, least privilege, seccomp, and workload-specific AppArmor or SELinux policy can materially reduce residual risk, but they do not eliminate kernel-level, control-plane, or misconfiguration risk [1]–[5].

The paper first builds the architectural context for Docker on Proxmox VE by explaining how virtualization and containerization interact and why containers, despite their efficiency benefits, do not provide a boundary equivalent to full virtualization when they share the host kernel [6], [7]. It then systematizes the kernel-level foundations of container hardening: namespaces, cgroups, capabilities, and seccomp. These mechanisms define visibility, resource consumption, privilege, and system-call reachability, but they do not solve vulnerable application logic, misuse of legitimate access, or kernel-level defects on their own [2], [3], [8]–[10].

The analysis then compares AppArmor and SELinux as Linux Security Module implementations. AppArmor is treated as a path-based, per-program confinement system, while SELinux is treated as a label- and type-based policy system. Their operational differences matter because container hardening depends not only on policy strength, but also on whether administrators can author, maintain, and troubleshoot the policy correctly for the workload under protection [11]–[15].

The practical contribution is the qualitative attack-scenario analysis in Chapter 6, supported by the public GitLab companion repository `cns-codeexamples` [16]. The `runc` CVE-2024-21626 scenario shows that vulnerable `runc v1.1.11` can create a host marker in the unconfined baseline, while a custom AppArmor profile blocks the same write path and fixed `runc v1.1.12` removes the vulnerable behavior. The remaining scenarios examine ABRT CVE-2025-12744 local privilege escalation, Redis CVE-2022-0543 Lua sand-

box escape, and Proxmox VE CVE-2024-21545 API/guest-agent trust-boundary abuse, mapping each case to detection indicators, countermeasures, and residual risk. The comparative matrix in Table 6.5 supports the broader conclusion that layered controls can reduce all four scenarios to low or low/moderate residual exposure, while control-plane and host-file-read issues remain especially sensitive because they involve trusted infrastructure interfaces.

Main Contributions

The main contributions of this paper are:

- it explains the interaction between Proxmox virtualization and Docker containerization, with particular attention to the different security properties of VM isolation and shared-kernel container isolation;
- it defines the precise security role and limits of namespaces, cgroups, capabilities, and seccomp in container runtime hardening;
- it compares AppArmor and SELinux as practical MAC enforcement systems for Docker workloads, including the implications of profile-based and label-based policy models;
- it applies these concepts to a four-layer Proxmox-based scenario analysis, supported by a public GitLab companion repository, showing how patching, seccomp, least privilege, AppArmor/SELinux enforcement, monitoring, and network restriction reduce residual risk across runtime escape, guest escalation, application sandbox escape, and management-plane trust-boundary scenarios.

Abstract

This paper examines how to harden Docker-based workloads deployed inside virtual machines on Proxmox VE by combining kernel-level isolation controls with Linux Mandatory Access Control enforcement. After positioning Proxmox and Docker within a layered virtualization-containerization architecture, the paper analyzes the main kernel mechanisms that underpin container confinement: namespaces, cgroups, capabilities, and sec-comp. These mechanisms reduce attack surface and blast radius, but they do not by themselves solve vulnerable application logic, misuse of already authorized access, kernel flaws, or operational misconfiguration [2], [4], [8]–[10].

The paper then studies Linux Security Modules as the enforcement framework for AppArmor and SELinux, emphasizing the operational trade-off between AppArmor’s path-based per-program profiles and SELinux’s label- and type-based confinement [11], [12], [14], [15]. Its practical contribution is a qualitative attack-scenario analysis on the four-layer stack shown in Figure 6.1, supported by a public GitLab companion repository and evaluated through a likelihood-impact risk model rather than measured exploit probabilities [16]. The scenarios cover runc CVE-2024-21626, ABRT CVE-2025-12744, Redis CVE-2022-0543, and Proxmox VE CVE-2024-21545. The comparative risk matrix in Table 6.5 shows that layered hardening can substantially reduce residual exposure, while control-plane interfaces, guest integration channels, vulnerable application logic, and shared-kernel assumptions remain important sources of residual risk.

Contents

Executive Summary	v
1 Introduction	1
2 Virtualization and Containerization	3
2.1 Traditional Virtualization	3
2.2 Containers	4
2.3 Differences between Virtualization and Containers	5
3 Platforms	9
3.1 Proxmox	9
3.2 Docker	9
4 Kernel-Level Isolation and Runtime Restriction	11
4.1 Namespaces	12
4.2 Control Groups (Cgroups)	14
4.3 Capabilities	16
4.3.1 Capability Sets	17
4.3.2 Applying Capabilities	18
4.3.3 Interaction with User Namespaces	19
4.3.4 Capabilities and the Root User	19
4.3.5 Dropping Capabilities and Least Privilege	19
4.3.6 Role in Container Security	20
4.4 System Call Filtering with seccomp	20
4.5 Summary: What These Mechanisms Do Not Solve	21
5 Mandatory Access Control and Linux Security Modules	23
5.1 Access control policies	23
5.1.1 Discretionary Access Control (DAC)	23

5.1.2	Mandatory access control (MAC)	25
5.1.3	Role-Based Access Control (RBAC)	26
5.1.4	Comparison of DAC, MAC, and RBAC	27
5.1.5	Best Practices for Access Control	28
5.2	Linux Security Modules (LSM)	29
5.2.1	LSM Hook-Based Enforcement	30
5.2.2	LSMs in Container Security	30
5.2.3	Relationship Between LSM, AppArmor, and SELinux	31
5.3	AppArmor	33
5.3.1	Role in the Threat Model	33
5.3.2	Mandatory Access Control Semantics	33
5.3.3	Path-Based Policy Model	33
5.3.4	Architecture and Policy Loading	34
5.3.5	Policy Expressiveness and Execution Transitions	34
5.3.6	Operational Modes and Profile Development	34
5.3.7	Security Value and Workload Scope	35
5.3.8	Limitations and Operational Trade-Offs	35
5.3.9	Illustrative Confinement Profile	36
5.3.10	Implications for Containerized Infrastructure	40
5.4	AppArmor in Container Security	41
5.4.1	Role of AppArmor in Container Isolation	41
5.4.2	Default Profiles and Workload-Specific Policy	41
5.4.3	Motivating Example: A Hardened Nginx Container	42
5.4.4	Operational Limitations	46
5.4.5	Summary and Transition to SELinux	46
5.5	Security-Enhanced Linux (SELinux)	46
5.5.1	What is SELinux?	46
5.5.2	How Does It Work?	47
5.5.3	Practical Introduction to SELinux	49
5.6	SELinux in Docker Containers	55
5.6.1	How Docker Uses SELinux	55
5.6.2	Enabling SELinux Support in Docker	56
5.6.3	Bind Mounts and Volume Labelling	57
5.6.4	Writing Custom Policies for Containers	58
5.6.5	The <code>--privileged</code> Flag and Its Implications	59

6	Attack Scenarios and Countermeasures	61
6.1	Chapter Overview	61
6.2	Methodology and Scenario Setup	61
6.2.1	Infrastructure Under Analysis	61
6.2.2	Scenario Construction Criteria	62
6.2.3	Risk Evaluation Model	63
6.3	Scenario 1: runc Container Escape via CVE-2024-21626 Contained by AppArmor	64
6.3.1	Scenario Description	64
6.3.2	Threat Model and Preconditions	64
6.3.3	Practical Implementation	65
6.3.4	Expected Impact	66
6.3.5	Detection Indicators	66
6.3.6	Countermeasures	67
6.3.7	Risk Analysis	68
6.3.8	Discussion	68
6.4	Scenario 2: ABRT Privilege Escalation via CVE-2025-12744 Contained by SELinux	68
6.4.1	Scenario Description	68
6.4.2	Threat Model and Preconditions	69
6.4.3	Practical Implementation	70
6.4.4	Expected Impact	70
6.4.5	Detection Indicators	71
6.4.6	Countermeasures	71
6.4.7	Risk Analysis	72
6.4.8	Discussion	72
6.5	Scenario 3: Redis Lua Sandbox Escape Contained by AppArmor	73
6.5.1	Scenario Description	73
6.5.2	Threat Model and Preconditions	73
6.5.3	Practical Implementation	74
6.5.4	Expected Impact	75
6.5.5	Detection Indicators	75
6.5.6	Countermeasures	75
6.5.7	Risk Analysis	76
6.5.8	Discussion	76
6.6	Scenario 4: Proxmox VE API Trust-Boundary Abuse via CVE-2024-21545	77

6.6.1	Scenario Description	77
6.6.2	Threat Model and Preconditions	77
6.6.3	Practical Implementation	78
6.6.4	Expected Impact	79
6.6.5	Detection Indicators	79
6.6.6	Countermeasures	80
6.6.7	Risk Analysis	81
6.6.8	Discussion	81
6.7	Comparative Risk Matrix	82
6.8	General Countermeasure Strategy	82
6.9	Chapter Summary	83
7	Conclusions	85
	Bibliography	87

List of Figures

2.1	On the left the high-level structure of a Type 1 hypervisor, on the right a Type 2 one	4
2.2	Architectural comparison between VMs and containers.	6
5.1	DAC's access to resource diagram.	24
5.2	MAC's access to resource diagram.	26
5.3	RBAC's access to resource diagram.	27
5.4	Basic operating principle of SELinux.	48
6.1	Four-layer stack used in the scenario analysis: Proxmox node, guest VM, LSM-enforced runtime, and container workload.	62
6.2	Guest-agent and Proxmox API trust-boundary abuse constrained by patching, API scoping, network policy, and monitoring.	81

Chapter 1

Introduction

Containerization technologies have become a fundamental component of modern IT infrastructures. Containers provide a lightweight and efficient method for deploying applications, allowing developers and system administrators to package software together with its dependencies and run it consistently across different environments. Platforms such as Docker and Podman have simplified development and deployment workflows, making container-based architectures common in cloud computing, microservices and many operational IT environments.

Despite their advantages in portability, scalability and resource efficiency, containerized environments introduce important security challenges. Containers share the same operating system kernel, which means that a vulnerability in the container runtime, an exposed management interface or a misconfigured privilege boundary may allow attackers to move beyond the intended container scope. For this reason, securing the runtime and enforcing appropriate isolation mechanisms is essential when deploying containers in production environments.

Hypervisors such as Proxmox VE provide an efficient way to manage infrastructure by enabling the deployment of virtual machines and containers within a controlled environment. When Docker containers are deployed inside virtual machines hosted on Proxmox VE, administrators can combine virtualization and containerization to improve flexibility and resource management. However, this layered architecture still requires explicit hardening to mitigate risks related to runtime vulnerabilities, guest compromise, management-plane exposure and operational misconfiguration.

This paper focuses on the hardening of Docker workloads running on Proxmox VE by analyzing the security mechanisms available at the virtualization, Linux kernel and container-runtime layers. In particular, it examines Linux Mandatory Access Control (MAC) systems such as AppArmor and SELinux as practical enforcement mechanisms

for restricting post-compromise behavior and strengthening workload isolation.

The objective is to analyze the security challenges of a Proxmox-based container infrastructure and present practical hardening strategies for Docker workloads. The work discusses the security properties and limits of virtual machines, containers, namespaces, cgroups, Linux capabilities, seccomp, AppArmor and SELinux, and then applies those concepts to controlled attack scenarios.

The implementation material referenced throughout the paper is published in the public GitLab companion repository `cns-codeexamples`. This repository supports reproducibility and allows readers to inspect the concrete implementation of the discussed security concepts without making the codebase the main object of the paper [16].

To provide the reader with the most comprehensive and direct understanding possible, logos and trademarks are included in the paper. **All trademarks and logos appearing in this document belong to their respective owners and are used for identification purposes only.**

Chapter 2

Virtualization and Containerization

This paper assumes familiarity with computer science, software engineering or related fields. The introductory sections therefore cover only the background needed to understand the security arguments developed in the rest of the paper.

2.1 Traditional Virtualization

Virtualization introduces an abstraction layer that decouples operating environments from the physical hardware beneath them. A single physical machine can therefore be partitioned into several independent virtual machines (VMs), each with its own operating system (OS) and application stack while sharing the same underlying hardware [17].

Some terminology is useful before comparing virtualization and containerization [18]:

- The host machine provides the physical resources, such as CPU, memory, storage and network interfaces, used by virtualized instances;
- A virtual machine, or guest, is the software instance running on the host with its own OS, virtual hardware and applications;
- A hypervisor mediates between VMs and the underlying physical hardware, allocating resources and keeping VMs isolated from one another. There are two main types:
 - Type 1, or bare-metal, hypervisors run directly on the host hardware. Examples include VMware ESXi, Microsoft Hyper-V, Xen, KVM and Oracle VM Server. They generally provide high performance and strong isolation, but require dedicated infrastructure and more complex management.

- Type 2 hypervisors run as applications on an existing OS. Examples include Oracle VirtualBox and VMware Workstation. They are easier to install and manage, but usually provide lower performance and depend on the security of the host OS [19].

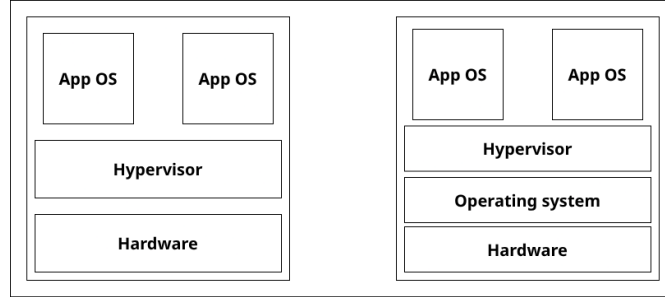


Figure 2.1: On the left the high-level structure of a Type 1 hypervisor, on the right a Type 2 one

- Kernel-based Virtual Machine (KVM) is a virtualization module in the Linux kernel that allows Linux to act as a hypervisor. Each VM has its own OS, kernel and virtual resources, with near-native performance on x86 hardware that supports Intel VT-x or AMD-V. KVM can run both Linux and Windows guests.
- Linux Containers (LXC) is an OS-level virtualization method that runs multiple isolated Linux systems on one control host using a shared Linux kernel. This reduces resource consumption, but it also means that LXC guests must be Linux-based.
- Quick Emulator (QEMU) is an open-source hypervisor that emulates a physical computer. The host sees QEMU as a user-space process with access to local resources, while the guest OS sees emulated devices and runs as if it were on physical hardware. QEMU can emulate components such as a motherboard, network controllers, storage controllers and serial ports [20].

2.2 Containers

Containers are lightweight executable units of software that package application code with its libraries and dependencies. They run in isolated user-space environments while sharing the host kernel, which supports deployment across desktop, traditional IT and cloud infrastructure [19], [21].

Unlike traditional VMs, containers rely on OS-level virtualization rather than hardware emulation. Their isolation and resource management are mainly provided by two

Linux kernel primitives: **namespaces** and **control groups (cgroups)**. These mechanisms allow multiple containers to run concurrently on the same host while limiting interference between applications and controlling access to system resources [22], [23].

Because containers share the host kernel, their security model differs from that of VMs: a kernel vulnerability can potentially affect all containers on the same host. Container hardening therefore often adds capability restrictions, mandatory access control frameworks such as SELinux or AppArmor, and system call filtering [6]. These mechanisms are examined in the following chapters.

2.3 Differences between Virtualization and Containers

This section examines the key differences between virtual machines and containers.

Architectural Differences

The primary distinction between virtual machines and containers lies in the layer of the computing stack that they virtualize. Traditional virtualization relies on a hypervisor, which abstracts the underlying physical hardware and enables multiple VMs to run on the same host. Each VM contains a full guest OS, its own kernel, virtualized hardware components and the application stack required to run services. This architecture allows different operating systems to coexist on the same physical host while maintaining strong isolation between instances [7].

Containers implement OS-level virtualization. Instead of virtualizing hardware, they share the host kernel and isolate applications in separate user-space environments. Each container packages only the application and its required libraries and dependencies, without a full guest OS; this makes containers lighter than VMs but also ties them to the host kernel [24]. Figure 2.2 summarizes this architectural difference.

As a consequence, containers typically cannot run a kernel different from the host, while VMs can run different operating systems simultaneously on the same hardware platform [25].

Resource Utilization and Performance

Virtual machines include a complete guest OS, kernel and system libraries. These components are replicated for each VM, increasing memory, storage and CPU consumption compared with containers.

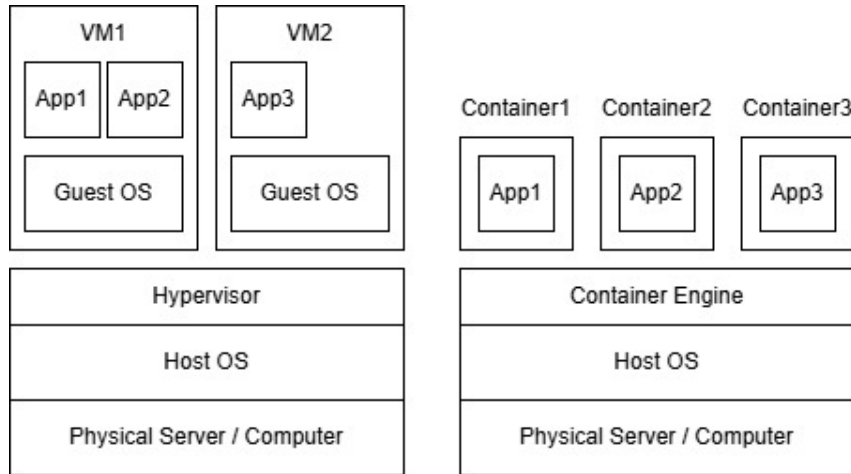


Figure 2.2: Architectural comparison between VMs and containers.

Containers do not require a full guest OS, which reduces their footprint and improves startup time. A VM must boot an entire operating system before executing an application, whereas a container starts as an isolated process on the already running host kernel.

Academic evaluations also report better results for container-based environments in most benchmarks, including memory performance, disk I/O, load benchmarks and compression tasks [22], [26], [27].

Isolation and Security

Isolation mechanisms differ significantly between virtual machines and containers, with direct security implications. Virtualization provides a distinct kernel for each VM and uses hardware-assisted isolation, making VMs suitable for workloads with strong security constraints, multi-tenant architectures or different operating-system requirements.

Containers provide process and namespace isolation through mechanisms such as namespaces and cgroups. These mechanisms offer strong logical separation, but they were designed for process control and resource management rather than as a complete substitute for VM isolation. Even when the *root* user inside a container has administrative control only within its namespace, a kernel vulnerability may allow one container to affect the host or other containers [6].

In practice, many systems combine the two solutions, deploying containers inside virtual machines. This layered approach provides both assurance and efficiency.

Portability and Compatibility

VMs rely on widely used virtualization technologies, many built on open-source components. They can be moved across systems that support compatible hypervisors, virtual

hardware and image formats.

Containers are usually more portable because application components and dependencies are packaged together in an image. Nevertheless, portability still depends on compatible container engines, CPU architectures, kernel features and operating-system expectations [28].

Chapter 3

Platforms

3.1 Proxmox

Proxmox Virtual Environment is an open-source server virtualization management platform built on two core technologies: QEMU/KVM for full virtual machines and Linux Containers (LXC) for lightweight OS-level virtualization. When available, QEMU uses KVM to take advantage of hardware virtualization extensions and improve performance. The platform runs on a Debian GNU/Linux base with a dedicated kernel derived from the Ubuntu kernel. It is licensed under the GNU Affero General Public License, version 3 (GNU AGPLv3). Proxmox VE can be used without a subscription, while optional subscription tiers provide enterprise repositories and support.

Proxmox VE can be managed through the web interface, command-line tools and REST API. Because it is Debian-based, administrators can also use the broader Debian package ecosystem, although production changes should remain compatible with the Proxmox-supported stack.

3.2 Docker

Docker is an open-source containerization platform used to build, distribute and run applications inside isolated execution environments called containers. Unlike full virtual machines, Docker containers share the host operating system kernel and rely on Linux kernel features such as namespaces, control groups and union file systems to provide process isolation, resource limitation and layered storage. This makes containers more lightweight than traditional virtual machines, allowing faster startup times and lower overhead. Docker images define the filesystem and dependencies required by an application, while registries such as Docker Hub or private repositories are used to store and distribute

them. The platform is commonly managed through the Docker CLI, Docker Compose, the Docker Engine API or higher-level orchestration systems. From a security perspective, Docker isolation should not be considered equivalent to hardware virtualization, since a compromised container still shares the host kernel. For this reason, mechanisms such as rootless mode, seccomp profiles, Linux capabilities, AppArmor or SELinux policies, image scanning and strict volume/network configuration are important when deploying containers in security-sensitive environments.

Chapter 4

Kernel-Level Isolation and Runtime Restriction

The previous sections introduced the architectural differences between traditional virtualization and container-based environments. While virtual machines achieve isolation through hardware abstraction provided by a hypervisor, containers rely on operating system-level mechanisms to isolate applications and regulate their interaction with shared kernel resources.

In Linux-based container systems, isolation, resource governance, privilege restriction, and runtime restriction are primarily implemented through four kernel mechanisms: *namespaces*, *control groups (cgroups)*, *capabilities*, and *seccomp*. Namespaces partition selected kernel resources, cgroups account for and constrain resource consumption, capabilities decompose traditional superuser privileges into finer-grained permissions, and seccomp filters the system calls available to a process. Together, these mechanisms allow multiple containerized workloads to run on the same host while maintaining process separation, limiting resource usage, reducing available privileges, and narrowing access to kernel entry points.

These mechanisms form the fundamental building blocks of modern container technologies such as Docker (see Chapter 3.2) and Linux Containers (LXC). The following sections describe how namespaces, cgroups, capabilities, and seccomp contribute to isolation, resource control, privilege restriction, and attack-surface reduction in containerized environments [2], [3], [8]–[10], [29], [30].

4.1 Namespaces

Linux namespaces are a kernel feature that provides isolation by partitioning selected global resources so that different groups of processes observe different views of the system. A namespace does not create a separate kernel; instead, it creates a distinct instance, or view, of a particular kernel resource for the processes that are members of that namespace [8].

The Linux kernel currently supports eight principal namespace types: cgroup, IPC, network, mount, PID, time, user, and UTS. Each type isolates a specific class of kernel resource. In particular, the time namespace virtualizes selected system clocks, whereas the cgroup namespace isolates the cgroup root directory visible to a process; it does not replace the accounting and limiting mechanisms implemented by cgroups themselves. Table 4.1 summarizes these namespace types.

Table 4.1: Linux namespace types and the resources they isolate

Namespace	Isolated resource	Description
Cgroup	Cgroup root directory view	Isolates the cgroup hierarchy root perceived by processes inside the namespace.
IPC	System V IPC and POSIX message queues ¹	Isolates inter-process communication objects such as shared memory segments, semaphores, and message queues.
Network	Network stack	Provides a separate set of network interfaces, addresses, routing tables, firewall rules, and sockets.
Mount	Filesystem mount points	Gives processes an independent view of mounted filesystems and mount propagation state.
PID	Process identifiers	Provides an independent process identifier space in which the first process acts as PID 1 for that namespace.
Time	Selected system clocks	Provides namespace-specific offsets for selected clocks, allowing processes to observe different clock values.
User	User and group IDs	Allows UID and GID mappings to differ between the namespace and its parent namespace.
UTS	Hostname and domain name	Isolates the system hostname and NIS domain name.

¹ Kernel mechanisms that enable inter-process communication.

Each namespace type contributes a different part of the container abstraction. For example, a newly created network namespace initially contains only a loopback interface; connectivity can then be established by linking it to other namespaces or to the host through virtual Ethernet devices. Similarly, processes running inside a PID namespace see only the processes that are visible from that namespace, and the namespace’s PID 1 process assumes special responsibilities for signal handling and reaping child processes.

Namespaces therefore enable multiple isolated environments to share a single Linux kernel while appearing, from inside each environment, to be separate systems. However, namespaces primarily partition visibility and authority over kernel resources; they do not determine how much CPU time, memory, I/O bandwidth, or task capacity a workload may consume. Quantitative resource management is provided by cgroups [8], [9].

4.2 Control Groups (Cgroups)

Control groups, commonly referred to as *cgroups*, provide kernel mechanisms for accounting, limiting, and prioritizing the resource usage of groups of tasks. While namespaces define which resources a process can observe or access, cgroups define how much of selected resources a workload can consume [9].

A cgroup is a node in a hierarchical organization of tasks to which resource policies can be applied. Each resource domain is managed by a controller, and the kernel exposes cgroup configuration through a virtual filesystem interface. Cgroups exist in two major versions, cgroup v1 and cgroup v2, which differ in hierarchy structure, controller naming, and some controller semantics. In cgroup v2, all enabled controllers share a single unified hierarchy, commonly mounted as `cgroup2`; available controllers are advertised through files such as `cgroup.controllers` [9], [31].

Table 4.2 presents cgroup v2 controllers that are particularly relevant to containerized workloads. Historical controllers such as `blkio`, `net_cls`, and `net_prio` belong to cgroup v1.

Table 4.2: Selected cgroup v2 controllers and managed resources

Controller	Managed resource	Example use in container environments
<code>cpu</code>	CPU scheduling and quotas	Regulate CPU time distribution and enforce CPU bandwidth limits for workloads.
<code>cpuset</code>	CPU cores and memory nodes	Restrict tasks to selected CPUs and memory nodes.
<code>memory</code>	Physical memory and swap	Enforce memory protection and limits, including reclaim and OOM handling behavior.
<code>io</code>	Block I/O	Control storage I/O weight, throttling, and latency-related behavior.
<code>pids</code>	Tasks, including processes and threads ¹	Limit task creation and mitigate uncontrolled fork or thread creation.
<code>rdma</code>	RDMA resources	Limit usage of Remote Direct Memory Access resources through controller-specific counters and limits.
<code>hugetlb</code>	HugeTLB pages	Limit usage of reserved huge page resources.
<code>misc</code>	Miscellaneous scalar resources	Account for and limit kernel-defined miscellaneous resources.

¹ The cgroup v2 documentation describes the controller in terms of tasks or TIDs. From the user’s perspective, the practical effect is to limit the creation of new processes and threads.

Some resource-control functions that were exposed as separate cgroup v1 controllers are represented differently in cgroup v2. Process freezing is provided by the `core cgroup.freeze` interface rather than by a separate `freezer` controller. Similarly, device access control is not exposed through the traditional cgroup v1 `devices` controller; in cgroup v2 it is implemented through cgroup-attached BPF programs¹ of type `BPF_PROG_TYPE_CGROUP_DEVICE`. The `rdma` controller, by contrast, is an independent cgroup v2 controller with its own semantics and files such as `rdma.max` and `rdma.current` [31].

The relationship between processes and cgroups must also be understood in terms of cgroup hierarchies. A process belongs to one cgroup within a given hierarchy. In cgroup

¹BPF is a safe, efficient, virtual-machine-like mechanism inside the Linux kernel that lets programs run at specific kernel hook points, especially for networking, tracing, and security. In modern Linux, “BPF” usually means eBPF, Extended Berkeley Packet Filter [32].

v2, however, several controllers can simultaneously apply resource policies to the same cgroup in the unified hierarchy. The kernel then accounts resource consumption for that cgroup and applies controller-specific enforcement when limits are reached [9], [31].

For example, when a cgroup exceeds a configured memory limit, the kernel may first attempt memory reclaim. If memory usage cannot be brought back within the configured limit, the kernel may invoke OOM handling scoped to the cgroup. In cgroup v2, the OOM killer does not select processes outside the cgroup that triggered the event; the concrete behavior also depends on settings such as `memory.oom.group`, which determines whether the workload should be treated as an indivisible unit. Some allocations may instead fail with `-ENOMEM` without immediately causing a process to be killed [31].

Through these mechanisms, cgroups support resource fairness and prevent individual services from monopolizing host resources. This is particularly important in multi-tenant container environments, where several workloads share the same kernel and hardware while requiring predictable limits on CPU, memory, I/O, and task creation.

4.3 Capabilities

Traditional Unix privilege management relied on a binary model in which processes running as the *root user* (*UID 0*) possessed broad system privileges, whereas unprivileged users did not. This model conflicted with the principle of least privilege², because a program often had to run with full administrative authority in order to perform only one privileged operation.

Linux addresses this limitation through *capabilities*, which divide traditional superuser privileges into finer-grained units. Each capability authorizes a class of privileged operations, allowing a thread or executable to receive only the privileges required for its intended function [10].

Examples of commonly used capabilities are shown in Table 4.3. Instead of granting full root privileges, the system can assign only the necessary capability to a process or executable. For instance, a service that must bind to a TCP port below 1024 can be granted `CAP_NET_BIND_SERVICE` without receiving unrelated administrative privileges.

²A security design principle according to which “every program and every user of the system should operate using the least set of privileges necessary to complete the job” [33].

Table 4.3: Linux capabilities and selected privileged operations [10]

Capability	Privilege granted
CAP_NET_ADMIN	Modify network configuration, including interfaces, routing tables, firewall rules, and traffic control.
CAP_NET_RAW	Create raw and packet sockets, for example for diagnostic tools or packet crafting.
CAP_SYS_MODULE	Load and unload kernel modules.
CAP_SYS_TIME	Change the system clock or hardware clock.
CAP_SYS_BOOT	Reboot or halt the system.
CAP_DAC_OVERRIDE	Bypass file read, write, and execute permission checks.
CAP_DAC_READ_SEARCH	Bypass file read permission checks and directory traversal restrictions.
CAP_SYS_ADMIN¹	Perform a broad set of system administration operations, including many mount, namespace, and quota-related operations.
CAP_SYS_PTRACE	Trace or debug processes subject to capability and access-control checks.
CAP_SYS_NICE	Change process priorities and scheduling policies.
CAP_SYS_RESOURCE	Override selected resource limits, such as the maximum number of open files.
CAP_SYS_CHROOT	Use the <code>chroot</code> system call to change the apparent root directory.
CAP_SETUID	Make arbitrary changes to process user IDs.
CAP_SETGID	Make arbitrary changes to process group IDs.
CAP_CHOWN	Change file ownership.
CAP_FOWNER	Bypass permission checks that normally require matching file ownership.
CAP_KILL	Send signals to processes owned by other users.
CAP_AUDIT_WRITE	Write records to the kernel audit log.

¹ **CAP_SYS_ADMIN** is intentionally highlighted because it is a highly overloaded capability. The Linux manual page warns against using it as a general substitute for new capability design; in container hardening, it should therefore be granted only when strictly necessary.

4.3.1 Capability Sets

In Linux, capability sets are associated with threads rather than with a process as a whole. In the modern model, five capability sets are relevant: permitted, effective, inheritable, bounding, and ambient. These sets determine both the privileges immediately exercisable by a thread and the way capabilities are transformed across an `execve()` call [10]. The sets are represented internally as bitmaps, where each bit corresponds to a specific capability. Table 4.4 summarizes their roles.

Table 4.4: Linux capability sets

Capability set	Flag	Description
Permitted	p	The capabilities that the thread may make effective; this set represents the upper bound of capabilities directly available to the thread.
Effective	e	The capabilities currently active and checked by the kernel when the thread performs privileged operations.
Inheritable	i	Capabilities that may be preserved across <code>execve()</code> and combined with file capability metadata for the newly executed program.
Bounding	b	The per-thread limit on capabilities that can be acquired during <code>execve()</code> ; since Linux 2.6.25, this set is no longer a system-wide attribute.
Ambient	a	A subset of capabilities preserved across <code>execve()</code> only for non-privileged files; if the executed file is privileged, for example set-user-ID, set-group-ID, or associated with file capabilities, the ambient set is cleared.

Among these sets, the *effective* set determines the privileges that a thread can actually exercise at a given moment. The other sets control which capabilities may be activated, inherited, preserved, or acquired when a thread replaces its program image.

4.3.2 Applying Capabilities

Capabilities can be assigned either to executable files or to running threads, enabling flexible privilege management.

Executable files may carry capability metadata stored as extended filesystem attributes. During `execve()`, the kernel computes the new permitted and effective sets by combining the file capability metadata with the calling thread’s inheritable, permitted, bounding, and ambient sets. It is therefore inaccurate to state that file capabilities are simply added unconditionally to the new program’s permitted and effective sets [10].

For example, the `setcap` utility can grant an executable the capability required to bind to privileged TCP ports without requiring it to run with full root privileges:

```
1 setcap cap_net_bind_service=+ep /usr/local/bin/myserver
```

In this example, `+ep` sets the permitted and effective bits in the file capability metadata for `CAP_NET_BIND_SERVICE`. When the executable is later invoked, the kernel applies the `execve()` transformation rules to determine the resulting thread capability sets [10], [34], [35].

Capabilities can also be manipulated at runtime by privileged threads through the

`capset()` system call or through libraries such as `libcap`. A common design pattern is to start a service with elevated privileges for initialization tasks, such as opening a privileged network port, and then drop all capabilities that are no longer required before processing untrusted input.

4.3.3 Interaction with User Namespaces

Capabilities interact closely with user namespaces, which allow user and group identifiers to be mapped differently inside a namespace than in its parent namespace. When a process creates or enters a user namespace, it may obtain a full set of capabilities inside that namespace without acquiring capabilities in the parent namespace or in the initial user namespace. These capabilities authorize privileged operations only over resources governed by non-user namespaces owned by that user namespace. Many global operations, such as changing the system time or loading kernel modules, remain reserved for processes that hold the relevant capabilities in the initial user namespace [10], [36].

4.3.4 Capabilities and the Root User

Under the Linux capability model, privilege checking is no longer purely binary. Nevertheless, UID 0 still receives special treatment in several kernel paths, particularly during `execve()`, unless securebits such as `SECBIT_NOROOT` are explicitly configured. A precise description is therefore that a process running with UID 0 normally receives a broad set of capabilities, but those privileges can be limited, removed, or redefined through capability sets, the bounding set, and securebits [10].

4.3.5 Dropping Capabilities and Least Privilege

The effect of dropping a capability depends on the set from which it is removed. Removing a capability from the effective set through `capset()` is reversible as long as the capability remains in the permitted set, because the thread may make it effective again. Removing it from the permitted set prevents the thread from directly reactivating it; except through a later `execve()` of a privileged file that is not masked by the bounding set, the capability will not become available again. Removing a capability from the bounding set through `prctl(PR_CAPBSET_DROP, ...)` is irreversible for the thread and its descendants [10].

This set-specific behavior supports the principle of least privilege. For example, a web server that needs to bind to port 80 can be granted only `CAP_NET_BIND_SERVICE` and then continue execution as a non-root user with no unrelated administrative capabilities.

If the service is compromised, the attacker should not automatically gain the ability to modify routing tables, mount filesystems, load kernel modules, or alter the system clock.

4.3.6 Role in Container Security

Capabilities play a critical role in container security by restricting the privileged operations that containerized processes can perform. Runtime systems such as Docker start containers with a restricted default allow list of capabilities, rather than with the complete privilege set normally associated with root on the host. This default configuration reduces privilege, but it is not a workload-specific minimum: Docker’s baseline retains capabilities that may be unnecessary for a given application. For this reason, container hardening should remove all capabilities that are not explicitly required by the application process, for example by using Docker’s capability drop and add options [1], [37], [38].

When combined with namespaces, cgroups, and mandatory access control systems, careful capability management helps reduce the impact of container compromise. It does not by itself provide a complete security boundary, but it is an essential part of a defense-in-depth strategy for hardened container deployments.

4.4 System Call Filtering with seccomp

seccomp (secure computing mode) is a Linux kernel mechanism for restricting the system calls that a process may invoke. It answers a different question from the mechanisms discussed above: namespaces determine what a process can see, cgroups constrain resource consumption, capabilities reduce privileged operations, and seccomp limits which kernel entry points remain available. Linux first exposed a coarse strict mode, but container hardening mainly uses seccomp-BPF, where a BPF filter evaluates each attempted system call and can allow it, deny it with an error, trap it for inspection, or terminate the task. Runtime systems and orchestrators rely on this filter mode because it is granular enough for real applications while still removing large parts of the kernel attack surface from ordinary workloads [1]–[3].

This matters especially in containers because every system call is a request to the shared host kernel. If a workload never legitimately needs to mount filesystems, create namespaces, trace processes, load packet filters, or alter identity-related state, denying those calls prevents the process from reaching those kernel paths at all. seccomp cannot undo an initial compromise, but it can narrow what compromised code may still ask the kernel to do: open files, create sockets, spawn processes, manipulate namespaces, or alter credentials [1]–[3].

For this reason, system-call filtering is widely used as a sandboxing primitive in browsers, language runtimes, containers, and other systems that execute risky code. In Docker and related runtimes, seccomp profiles are usually allow lists: they encode which kernel interfaces the workload actually requires rather than declaring the workload safe in the abstract. A conventional service process rarely needs mount operations, `unshare()`, `setns()`, `ptrace()`, or calls that alter identity or execution context. Docker’s default seccomp profile follows this model by preserving broad compatibility while denying many higher-risk or uncommon system calls [2], [3], [37].

The security contribution of seccomp is therefore attack-surface reduction at the kernel boundary. It complements namespaces, cgroups, and capabilities rather than replacing them: a process may be isolated and stripped of privilege while still having access to kernel entry points its application logic does not need. Conversely, seccomp may deny a call even when DAC permissions or Linux capabilities would otherwise make it meaningful. It is also orthogonal to AppArmor and SELinux: seccomp decides whether a class of kernel request may be made, whereas MAC systems decide whether specific files, paths, sockets, mounts, signals, labels, or execution transitions are permitted once that kernel path is reached [2], [3], [8]–[10].

seccomp policies are nevertheless difficult to design correctly. Real programs depend on system calls through libraries, runtimes, allocators, threads, and kernel fallbacks; the same high-level action may also map to different calls across architectures or software versions. An overly permissive profile leaves unnecessary kernel functionality exposed, while an overly narrow one breaks legitimate execution and encourages broad exceptions. Even seccomp-BPF reasons only over system-call numbers and selected raw arguments: it can observe `openat()`, `clone()`, or `ioctl()`, but not the business intent behind them. It is strongest when combined with the other mechanisms in this paper and derived from a specific workload model [1]–[3], [37].

4.5 Summary: What These Mechanisms Do Not Solve

The mechanisms discussed in this chapter substantially improve container security, but their value must be stated precisely. Namespaces partition visibility, cgroups constrain resource consumption, capabilities reduce ambient privilege, and seccomp narrows system-call reachability. Together they reduce attack surface, raise exploitation cost, and limit blast radius after compromise. They do not prove that an application is safe, eliminate compromise, or make a shared-kernel container equivalent to a virtual machine; host

security still depends on kernel, runtime, and configuration correctness [2]–[4], [6], [8]–[10].

Most importantly, these mechanisms do not fix vulnerable application logic. Authentication flaws, injection bugs, unsafe deserialization, insecure administrative interfaces, and incorrect authorization remain exploitable inside an isolated container. The same applies to malicious or compromised supply-chain components: an image, package, plugin, or dependency can remain inside its sandbox while exfiltrating legitimately available data, credentials, or network access. Isolation and system-call filtering constrain post-compromise behavior, but they do not prove that the code inside the boundary is trustworthy [1], [2], [4], [5], [38].

They also do not solve misconfiguration, which is often the practical failure point. Excessive capabilities, writable host bind mounts, host namespace sharing, privileged mode, disabled seccomp filters, unconfined execution, and broad exceptions can collapse the intended boundary even when the features are nominally enabled. Administrators must still define which paths, network communications, identities, capabilities, and system calls are genuinely required. If that boundary is too broad, the sandbox is weak; if it is too narrow, operations break and policy is often relaxed until it stops providing meaningful containment [1]–[4], [33], [37].

A further limitation is that these controls do not prevent misuse of already authorized access. A compromised process may still read mounted secrets, steal tokens from memory, abuse an allowed internal API, or use a control-plane credential available for normal operation. This is where lateral movement becomes relevant: if the workload can legitimately reach a database, message broker, registry, Docker socket, or management API, compromise may propagate through those trusted channels. Isolation reduces the scope of that movement; it does not eliminate it [1], [4], [5], [38].

Finally, these mechanisms do not remove kernel-level risk. Containers still share the host kernel, and a reachable kernel vulnerability remains dangerous even when namespaces, capabilities, cgroups, and seccomp are active. Runtime flaws, LSM bypasses, driver bugs, and other low-level defects may still undermine the model. These controls should therefore be treated as damage-reduction mechanisms within a broader security program, not as self-sufficient guarantees. They complement, but do not replace, secure design, code review, image provenance, timely patching, monitoring, logging, least-privilege review, credential hygiene, and operational security [1]–[5].

Chapter 5

Mandatory Access Control and Linux Security Modules

5.1 Access control policies

Access control is the foundational decision engine of any security architecture, determining whether a given subject is permitted to perform a specific action on a given resource under a defined set of conditions. Beyond a simple login gate, it governs the permitted actions and degree of control that an entity holds over system assets, making it central to confidentiality, integrity and availability. Its scope extends into authorization, where misconfigured or overly broad permissions can transform a single compromised account into a broader breach. This risk is especially relevant in cloud and SaaS environments, where IAM policies, sessions and tokens often serve as the primary proof of identity. Another recurring problem is weak permission lifecycle management: access is often granted quickly but revoked slowly, leaving residual privileges behind. This chapter introduces DAC, MAC and RBAC before focusing on AppArmor and SELinux as Linux MAC implementations [39].

5.1.1 Discretionary Access Control (DAC)

DAC is an identity-based access control model in which owners of the resources decide who can access them and what privileges to grant. Permissions for each user or group are stored in an Access Control List (ACL), which acts as an enforced security policy for that resource and cannot be overridden by regular users. These are the main points regarding DAC:

- Identity verification: access is granted based on identity, typically verified through

a username-password combination or any other required authentication method;

- Lower security compared to MAC: since access decisions rely on the judgement of individual resource owners, DAC is inherently less secure than Mandatory Access Control (MAC). Attackers who compromise a user account can exploit its permissions to spread malware or escalate privileges. Common risks include privilege creep, excessive resource sharing, and users inadvertently granting access to unauthorized parties;
- Maintenance overhead: DAC requires resource owners to keep their ACLs continuously updated. As organizational roles and network structures evolve, ACLs can quickly become outdated, potentially retaining access for former employees or failing to reflect role changes;
- Resource opacity: object attributes such as size, name, and directory path remain invisible to unauthorized users;
- Transferable ownership: users can transfer ownership of objects to other users, making access easy to share or delegate. However, a user cannot grant permissions that exceed their own [40] [41] [42].

When to use DAC

DAC is not suitable for organizations that handle highly sensitive or classified data such as governments' institutions, medical records and financial informations because it cannot reliably enforce the principles of least privilege and need-to-know. However, it enables a functional infrastructure that is not overly complex so it can be a good choice for small businesses with limited budgets and IT professionals [43].

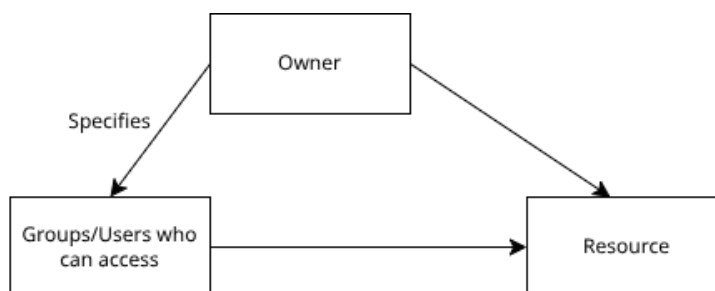


Figure 5.1: DAC's access to resource diagram.

5.1.2 Mandatory access control (MAC)

Mandatory Access Control (MAC) takes a fundamentally different approach from DAC by removing access decisions from individual users entirely. The objective was to enable finer-grained security policies that extend beyond the conventional DAC permission model based on owner-group and read/ write/execute rights. This two-tier enforcement model positions MAC not as a substitute for traditional permissions but as a complementary layer. Every system call and every access attempt is subject to policy verification. Rather than replacing the permission bits managed by tools such as `chmod` or `chown`, MAC operates alongside them in a defined sequence, adding a deeper and non-discretionary level of control, regardless of user privileges, including those of the root account. Indeed, all permissions are defined by system administrators and enforced by the operating system or security kernel: users cannot modify or transfer access rights, even for resources they created themselves. MAC can be summarised as follows [43] [39]:

- Policy-driven enforcement: access is governed by centrally defined security policies that are strictly enforced at the system level, leaving no room for user discretion;
- Strict compartmentalization: administrators classify users into subsets and assign security attributes and clearance levels to both users and resources, limiting each subset's exposure to only what is necessary;
- Implementation complexity: MAC is more complex to implement and maintain than DAC. Security levels and clearances must be manually configured and kept up to date requiring constant attention from admins;
- Ongoing administration: every new user and every new piece of data requires administrators to update object and account configurations accordingly;
- Explicit access requests: users must explicitly request access to each new resource, as no access is granted implicitly.

The typical access flow under the combination DAC-MAC (they are usually used together, except for really high-threat models) works as follows:

1. The administrator defines security policies and assigns attributes to every user or processes that access data and objects (files, databases, network ports, etc.), including confidentiality levels and clearances for each project or resource;
2. When a subject (no distinction between process and user) attempts to access a resource, the kernel first evaluates the standard DAC permissions;

3. If DAC deny the request, the operation is rejected and SELinux is never consulted;
4. If DAC grants the request, the control is passed to the system to check the SELinux policy;
5. The operating system compares the security attributes of both parties and determines whether access should be granted. Holding a high clearance level alone is not sufficient: a user classified as top secret will still be denied access to a file if they do not also belong to the required category for that object. For example, a file containing engineering documents may be accessible only to users who simultaneously hold a top secret clearance and are members of the engineering category. Users meeting both conditions can also access resources that require a lower clearance level;
6. The user provides their credentials to complete the authentication step and if all attributes match the policy, access is granted.

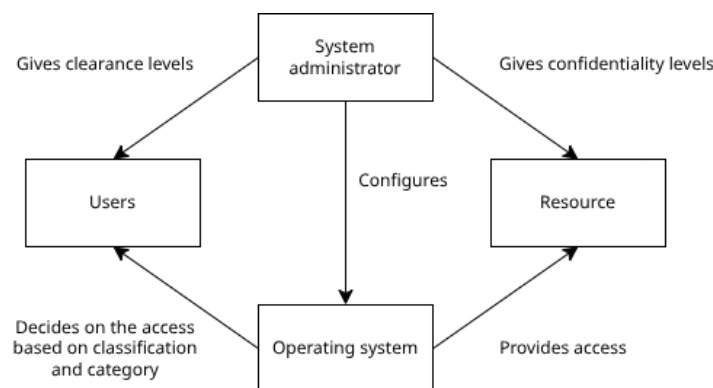


Figure 5.2: MAC's access to resource diagram.

5.1.3 Role-Based Access Control (RBAC)

In every company, a lot of users have similar responsibilities and duties, therefore require similar access rights. Rather than configuring specific permissions for each person individually, administrators define distinct roles within the organization and determine which resources every role should be able to access. Finally it assigns users to those roles accordingly. RBAC is the most widely adopted access control model and a natural fit for implementing the principle of least privilege. Admins can further refine a RBAC system by assigning users to multiple roles simultaneously (with caution!), defining role hierarchies that reflect different levels of responsibility and establishing constraints that encode specific business rules [44] [39]. The main advantages and disadvantages of this policy are the following:

- Simplified onboarding: since all required permissions come pre-bundled within a role, new team members can be granted appropriate access immediately, reducing setup time and minimizing the risk of configuration errors;
- Ease of maintenance: with well-defined roles, day-to-day administration is reduced to routine onboarding, offboarding and role reassignment operations;
- Centralized, consistent policies: security teams can define and enforce uniform RBAC policies across the entire organization, eliminating the inconsistencies that arise from discretionary access models;
- Reduced risk exposure: because users only have access to the resources their role justifies, the potential attack surface is significantly narrowed;
- Complex initial deployment: in large organizations, mapping the full web of responsibilities and interdependencies into well-defined roles is a non-trivial task;
- Security vs. simplicity trade-off: more granular roles improve security but increase administrative overhead and vice versa;
- Risk of over-privileging: assigning users to too many roles or defining roles that are too broad can result in users accumulating permissions that exceed their actual responsibilities. The typical example appears when an organizations scale and consequently employees often accumulate roles over time without obsolete ones being removed. This not only broadens the attack surface but can also create compliance issues if access rights are not periodically reviewed and revoked.

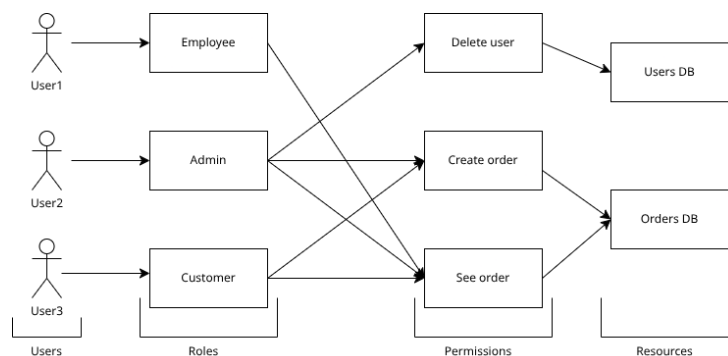


Figure 5.3: RBAC's access to resource diagram.

5.1.4 Comparison of DAC, MAC, and RBAC

Feature	MAC	DAC	RBAC
Who controls access?	Admins via mandatory OS-enforced policies.	Resource owner.	Roles defined by admins.
Who sets permissions?	Only system admins can define security labels and policies.	Resource owner.	Role configurations set by admins define user permissions.
Can users change their permissions?	No, not even their own.	Yes, freely.	No, only administrators can update roles.
Security level	The highest, it ensures strict data confidentiality and integrity.	The lowest: it can lead to privilege escalation or data exposure.	Offers a strong balance of security and usability by following the principle of least privilege.
What access is based on?	Labels and classifications.	Resource ownership and the owner's discretion over sharing.	Roles and responsibilities within the organization.
Root account privileges	Even root can be constrained by mandatory policies.	Root privileges are broad unless restricted by other controls.	It may exist as a user but its privileges depend on role assignments: unrestricted access requires explicit role grants.

Table 5.1: Comparison of MAC, DAC and RBAC access control models

5.1.5 Best Practices for Access Control

Implementing effective access control requires following a set of established best practices [43] [39] [45]:

- Regularly review and update access policies: access policies should be periodically reassessed to adapt to changes in the threat landscape, regulatory requirements, business processes and user roles. All key stakeholders (IT professionals, employees, business owners, ecc.) should be involved in the reviews to be sure that nothing is left uncovered and to aligned with business' goals;
- Implement multi-factor authentication (MFA): MFA requires users to provide more than just one verification factor such as a password combined with a hardware se-

curity token or a biometric trait. This significantly reduces the risk of unauthorized access even in cases of credential theft or phishing attacks;

- Adopt password-less authentication or ensure strong password policies: passwords should be long and complex but organizations should take into account password-less methods such as the ones mentioned in the previous point;
- Avoid shared accounts: each user should have an individual account to ensure traceability and accountability. Where shared access is unavoidable, privileged access management (PAM) solutions should be used to monitor and log sessions;
- Apply the principle of least privilege: users should be granted only the minimum level of access required to perform their duties;
- Adopt a zero trust paradigm: following the principle of never trust, always verify. Even if a request comes from inside the organization's network, it must be authenticated and authorized;
- Monitor and audit access control systems: access logs and audit trails should be regularly reviewed to detect unauthorized access attempts and identify possible security gaps. If any problems or incidents arise, you can then analyse them to better understand the cause and consider a possible solution.

5.2 Linux Security Modules (LSM)

Linux Security Modules (LSM) provide the kernel-level framework through which Linux hosts security modules that mediate access to kernel objects and security-sensitive operations. LSM is not itself a policy language or a single security policy; it is the in-kernel mediation layer that lets modules such as AppArmor and SELinux attach checks to relevant execution paths and apply mandatory decisions at runtime. This distinction matters because AppArmor, SELinux, and other LSM implementations use the same kernel framework while expressing and enforcing policy through different models [11], [12], [14], [15].

In containerized Linux environments, the practical significance of LSM is that enforcement occurs inside the kernel shared by all containers on the host. Namespaces isolate visibility, cgroups constrain resource consumption, capabilities decompose root privilege, and seccomp restricts available system calls; LSM adds policy-driven authorization that can deny or audit operations even after a process has reached the kernel and even when ordinary Unix ownership and mode bits would otherwise allow the action [2]–[4], [8]–[10].

5.2.1 LSM Hook-Based Enforcement

LSM enforcement is hook-based. The kernel exposes security hooks at points where a subject is about to perform an operation on a security-relevant object or subsystem, and the loaded security module evaluates the request before the kernel proceeds. These hooks cover program execution, file creation and opening, file attribute changes, socket operations, inter-process communication, and capability checks; representative examples include `execve()` transitions, file open and truncate operations, socket connect and send operations, System V message-queue operations, and capability mediation [10], [11].

This placement is what makes LSM-based MAC different from traditional DAC. DAC decisions are principally derived from ownership and permission bits, while MAC policy is centrally defined and enforced by the operating system. A process may therefore be denied by AppArmor or SELinux even when the relevant UID, GID, and mode bits would allow the operation. In Linux, this narrows the effective authority of both ordinary users and privileged processes, including processes running as root [12], [14], [15], [39], [43].

The same hook-based design also supports auditing. An LSM implementation can permit an operation, deny it, or emit audit records describing attempted policy violations. This is important during policy development because administrators need to observe what was blocked, or what would have been blocked, before enforcing a profile or policy. AppArmor’s complain and enforce modes illustrate this workflow, while SELinux likewise records policy denials as part of its operational model [15], [46]–[48].

5.2.2 LSMs in Container Security

LSMs play an important but bounded role in container security. Containers do not emulate a separate kernel in the way a virtual machine does; they depend on the host’s Linux kernel for process management, filesystems, networking, and access control. In the four-layer stack considered in this paper, consisting of the Proxmox host, the guest VM, the LSM-enforced runtime, and the container workload, LSM enforcement is therefore most relevant at the guest-kernel layer, where container processes are ultimately mediated [3], [4], [6].

For that reason, LSMs should be understood as a defense-in-depth layer, not as a self-sufficient isolation boundary. Docker frames container security around namespaces, cgroups, capability restrictions, daemon attack surface, and additional kernel hardening features; Kubernetes likewise treats seccomp, AppArmor, SELinux, capability management, and privileged-mode restrictions as interacting controls rather than interchangeable substitutes [1]–[3], [37], [38], [49].

In concrete terms, AppArmor and SELinux can narrow the blast radius of a compromised container by constraining what a post-compromise process may still do. An AppArmor profile can restrict which filesystem paths a containerized process may read, write, or execute, as well as which network, mount, ptrace, and signal operations remain available. Docker’s default `docker-default` AppArmor profile and its workload-specific profile examples demonstrate this model directly: the profile remains enforced for the container process, can block operations such as raw-socket use or mount attempts, and can be replaced with a tighter custom policy when necessary [13], [49]–[51].

SELinux provides analogous value through labels and type enforcement. Red Hat’s container policy guidance emphasizes that container workloads run in SELinux domains and can be assigned more tailored types when the generic container domain is too strict or too permissive for the workload. This is particularly relevant for access to host-mounted storage, device nodes, and network-facing behavior, because the policy decision is made over process and object labels rather than only over pathnames. Kubernetes documentation similarly notes that SELinux labels can restrict a container from accessing resources outside the intended container scope and that these MAC controls can extend protection beyond the container’s immediate userspace boundary [3], [15], [49], [52].

At the same time, LSMs do not replace namespaces, cgroups, seccomp, capabilities, or read-only filesystem controls, and they do not make kernel vulnerabilities irrelevant. A vulnerable or overly privileged runtime configuration can still weaken the overall posture, and Kubernetes explicitly warns that privileged containers can override or undo seccomp, AppArmor, and SELinux restrictions. Accordingly, LSMs are best treated as one layer in a broader hardening strategy, not as a complete container security solution [2]–[4], [37], [49].

5.2.3 Relationship Between LSM, AppArmor, and SELinux

The relationship between these terms must be kept precise: LSM is the kernel framework, while AppArmor and SELinux are concrete LSM implementations that enforce MAC policies through that framework. Linux therefore does not “run LSM policy” in the abstract; it runs one or more security modules that plug into the LSM interface and make decisions at the relevant hooks [11], [12], [14], [15].

Although both AppArmor and SELinux implement MAC, they differ substantially in policy model. AppArmor is primarily profile-based and path-oriented: policies attach to programs and commonly describe permitted access through executable paths, file paths, capabilities, network rules, mount rules, ptrace rules, and signal rules. SELinux is primarily label-based: every process and system resource has a security context, and everyday

enforcement is usually driven by type rules that define which subject types may interact with which object types. Path-oriented confinement is often easier to read and prototype, while label- and type-oriented confinement tends to provide stronger semantic stability across rename and bind-mount scenarios when labeling is correct and consistently maintained [12]–[15], [46].

These model differences also affect deployment. AppArmor is common on Ubuntu-based systems and is often attractive where administrators prefer per-application profiles and direct policy authoring. SELinux is the standard MAC system in Red Hat Enterprise Linux documentation and tooling, and is commonly preferred where label integrity, domain separation, and richer policy composition are central requirements. Neither approach is universally better in the abstract; the relevant question is which model fits the environment’s operational needs, administrative expertise, and workload characteristics [15], [46], [47].

Table 5.2 summarizes the high-level distinction between the two LSM implementations [12]–[15], [50].

Aspect	AppArmor	SELinux
Underlying role	LSM implementation	LSM implementation
Primary policy model	Per-program profiles; predominantly path-based rules	Label/security-context model; predominantly type-enforcement rules
Typical policy objects	Executable paths, file paths, capabilities, network, mount, signal, and ptrace rules	Subjects and objects identified by security contexts and types
Administrative tendency	Often operationally simpler for narrowly scoped application confinement	Often more structured and expressive for large, label-centric system-wide confinement
Container use	Runtime default or custom profiles can confine container processes	Container domains and custom types can confine container processes and labeled resources

Table 5.2: High-level comparison between AppArmor and SELinux.

From the perspective of this paper, AppArmor and SELinux solve a similar class of problems, but they do so through different policy abstractions, operational workflows, and trade-offs. This is why the following sections examine them separately in the context of containerized production environments [3], [4], [50].

5.3 AppArmor

5.3.1 Role in the Threat Model

For the threat model introduced, AppArmor adds kernel-enforced confinement on top of the namespace, cgroup, capability, and seccomp mechanisms discussed in Chapter 4. Its role is not to prevent the initial software flaw, but to restrict what a compromised host service or container workload may still read, write, execute, mount, signal, or trace. Before applying AppArmor to Docker workloads, it is therefore useful to understand it as a general Linux confinement mechanism rather than merely as a container option [12], [13], [46], [47].

5.3.2 Mandatory Access Control Semantics

AppArmor (Application Armor) is a Linux Security Module (LSM) that implements Mandatory Access Control (MAC) through per-program security profiles. Profiles are defined in user space, loaded into the kernel, and associated with running tasks; processes for which no profile applies run in an unconfined state equivalent to ordinary Linux discretionary access control (DAC) alone [12].

AppArmor does not replace Unix ownership and permission bits. It supplements them with a second, non-discretionary decision layer, so a process may satisfy DAC and still be denied if the requested operation falls outside the active profile. Conversely, if AppArmor is disabled or a task is unconfined, only DAC remains in effect [12], [46].

5.3.3 Path-Based Policy Model

A defining property of AppArmor is that it is path-based. Policies refer to file-system path patterns, not to persistent inode¹ labels. In operational terms, a profile states which pathnames a program may read, write, execute, link, or map into memory, and AppArmor resolves the path at access time [13], [46].

This differs from both DAC, which centers on user and group ownership, and label-based MAC systems such as SELinux, which reason over security labels and type-enforcement rules attached to subjects and objects [13], [14], [52]. The path-oriented model is often intuitive because rules mirror an application’s deployed file layout, but correctness then depends on stable pathname resolution and predictable mount layouts.

¹In Linux, an inode (short for “index node”) is a data structure that stores information about a file or directory on a file system, such as file type, file permissions, file size, timestamps, pointers to the blocks on the disk where the file’s data is stored [53].

5.3.4 Architecture and Policy Loading

Architecturally, AppArmor consists of kernel-side LSM enforcement, a user-space profile language and parser, and administrative tools for loading, inspecting, and tuning policies. Profiles are ordinary text files, conventionally stored under `/etc/apparmor.d/`, compiled by `apparmor_parser`, and enforced at predefined LSM hook points. When a process reaches such a hook, the kernel identifies the active profile, compares the requested access with the policy, and allows or denies the operation [12], [13], [47].

On contemporary Ubuntu systems, `systemd` participates in early policy loading through the compiled policy cache, while the regular `apparmor.service` unit manages profile loading and reloading during system operation [46].

5.3.5 Policy Expressiveness and Execution Transitions

The profile language is broader than the simplified “files only” view often found in introductory descriptions. In addition to file and directory path rules, AppArmor policies can mediate Linux capabilities, networking, mount operations, UNIX sockets, DBus interactions, signals, ptrace, user namespaces, and execution transitions across `execve()` [13].

Execution transitions are especially important. Modes such as `ix`, `px/Px`, `cx/Cx`, and `ux/Ux` determine whether a helper program inherits the current profile, enters another profile, enters a subprofile, or executes unconfined. In a hardened profile, transitions that escape confinement must be used deliberately and sparingly [13].

5.3.6 Operational Modes and Profile Development

Operationally, the most important profile states are *enforce* and *complain*. Enforce mode blocks operations outside policy and records the violation; complain mode logs the same events without blocking them, which makes it useful for development and regression testing. If the profile is unloaded, disabled, or never applied, the process runs unconfined [46], [47].

Ubuntu’s documentation treats profile development as a gradual workflow: start in complain mode, exercise realistic workloads, review denials with tools such as `aa-logprof`², and switch to enforce mode once legitimate access patterns are understood [47]. This gives administrators a practical path from observation to least privilege.

²The program `aa-logprof` can be used to scan log files for AppArmor audit messages, review them and update the profiles [47].

5.3.7 Security Value and Workload Scope

From a security standpoint, AppArmor supports least privilege by turning application behavior into an explicit allowlist. A network daemon can be limited to a specific content tree, one log file, one socket family, and only the capabilities required for its function. If the daemon is exploited, the attacker inherits a process that remains constrained by the profile.

The security value is therefore post-compromise containment: AppArmor does not remove the initial software flaw, but it can bound what happens after that flaw is reached [12], [13], [33], [54]. Because enforcement is policy-based rather than signature-based, it can also contain previously unknown vulnerabilities when malicious behavior requires access outside the permitted profile [54].

This model is useful for several workload classes. Server processes can be limited to their configuration, content, log, socket, capability, and signal requirements; desktop-facing software can be constrained against document-parser or browser compromise; and Ubuntu uses AppArmor as a core confinement mechanism for snaps and Ubuntu Core [46]. For container hosts, the same model complements namespaces and cgroups by constraining container payloads from the host kernel's point of view.

AppArmor is also practical where profile authoring and maintenance must remain approachable: profiles are plain text, tied to executable paths, and can reuse common abstractions, variables, tunables, and local override files [13], [47]. This does not make it universally simpler, but it helps explain its operational appeal on Debian and Ubuntu-like systems with stable package-managed path layouts.

5.3.8 Limitations and Operational Trade-Offs

The same properties also define AppArmor's limitations. Because mediation is pathname-based, policy meaning depends on how objects are reached through the file-system and mount hierarchy. Renaming an application directory, introducing a bind mount under a new path, changing namespace-relative mount points, or relocating cache and log directories can alter rule matching even when the underlying file contents remain the same.

AppArmor profiles must therefore be maintained together with the deployed file-system layout. Alias rules and flags such as `chroot_relative` and `attach_disconnected` can address some path-layout issues, but the documentation warns that `attach_disconnected` is unsafe and may introduce aliases to objects that should not be allowed [13]. Profile drift is a real concern: updates, plugins, helper binaries, or relocated storage paths can either break legitimate behavior or tempt operators toward coarse wildcards [13], [47].

A second limitation is policy completeness. A narrow profile is useful only if it captures legitimate access accurately; otherwise, administrators may weaken it after repeated denials. Conversely, broad rules such as `/var/** rw` or unconfined execution transitions may preserve functionality while providing little confinement [13].

This is where the contrast with SELinux is useful. SELinux type enforcement is less tightly coupled to pathnames and file-system relocation, whereas AppArmor remains easier to read when the deployment layout is known and stable [13], [14], [52]. The difference is a design trade-off between path-oriented operational convenience and label-oriented invariance, not a categorical ranking of one mechanism over the other.

5.3.9 Illustrative Confinement Profile

A realistic example helps to make these properties concrete. Consider a small host-level example built around a minimal C HTTP service called `docviewd`. A GitLab companion implementation is cited for this example [16]. Unlike the container examples discussed later, this program is compiled and installed directly on an Ubuntu guest machine, for example at `/usr/local/bin/docviewd`, and is confined as an ordinary host process. The daemon listens on `127.0.0.1:8080`, serves static content from `/srv/docview/public/`, writes an access log under `/var/log/docviewd/access.log`, stores its PID file at `/run/docviewd/docviewd.pid`, and runs as a dedicated non-root user, `docview`.

The intended behavior is narrow: the daemon should read the public document tree, append to its log, update its PID file, accept ordinary IPv4 TCP connections, and report its AppArmor label through `/proc/self/attr/current`. It should not read a sibling file outside the public root, write to `/tmp`, mount filesystems, trace other processes, or execute a shell. The routes `/forbidden-read`, `/forbidden-write`, and `/forbidden-exec` are policy probes, not application features; the DAC permissions are chosen so that denied operations can be attributed to AppArmor rather than ordinary Unix ownership alone.

The custom profile is installed as `/etc/apparmor.d/usr.local.bin.docviewd`, loaded with `apparmor_parser`, and attached to executions of `/usr/local/bin/docviewd`. The profile name follows the companion repository convention, `docviewd-demo`, so that denial records are easy to filter in audit or kernel logs. A profile implementing this confinement is shown below.

```
1 #include <tunables/global>
2
3 profile docviewd-demo /usr/local/bin/docviewd flags=(audit) {
4     #include <abstractions/base>
5 }
```

```

6  network inet stream,
7
8  /usr/local/bin/docviewd mr,
9
10 /srv/docview/public/ r,
11 /srv/docview/public/** r,
12
13 /var/log/docviewd/ rw,
14 /var/log/docviewd/access.log ak,
15
16 /run/docviewd/ rw,
17 /run/docviewd/docviewd.pid rwk,
18
19 @{PROC}/self/attr/current r,
20 owner @{PROC}/@{pid}/attr/current r,
21
22 signal (receive) peer=unconfined,
23 deny ptrace (trace),
24 deny mount,
25 }

```

Listing 5.1: Custom AppArmor profile for docviewd.

Profile Interpretation

The declaration `profile docviewd-demo /usr/local/bin/docviewd` gives the profile a readable name while attaching it to the executable path used on the host. The `audit` flag is enabled for visibility during profile development, although broad auditing would normally be too verbose for production services. Once the profile is loaded, new executions of `/usr/local/bin/docviewd` enter the `docviewd-demo` domain automatically [13], [47].

Only `abstractions/base` is included because the service needs ordinary runtime support but no DNS or user lookup in the request path. The network rule permits the IPv4 stream socket used by the daemon, while loopback-only exposure remains an application and deployment choice. The file rules permit the binary to be read and mapped, the public tree to be read, the log to be append-only, and the PID file to be updated at startup.

The profile deliberately contains no rule for `/srv/docview/secret.txt`, no rule for `/tmp/**`, and no execute rule for `/bin/sh`, `/usr/bin/dash`, or other generic helpers. Reading `/proc/self/attr/current` is allowed for the status endpoint, signal receipt from an unconfined peer allows normal administrative shutdown, and explicit denials for tracing and mounting document that the service has no legitimate reason to inspect other

tasks or create new mount relationships [12], [13].

Deployment Workflow

On an Ubuntu host with AppArmor enabled, the standard deployment sequence is to compile the daemon, install the files with DAC permissions that would otherwise allow the probes, load the profile in observation mode, request both allowed and forbidden paths, and finally switch the profile to enforcement. The following commands summarize the essential operations from the GitLab companion implementation.

```
1 mkdir -p build
2 gcc -O2 -Wall -Wextra -o build/docviewd src/docviewd.c
3
4 id -u docview >/dev/null 2>&1 || sudo useradd \
5     --system --home-dir /nonexistent \
6     --no-create-home --shell /usr/sbin/nologin docview
7
8 sudo install -m 0755 build/docviewd /usr/local/bin/docviewd
9 sudo install -d -m 0755 /srv/docview/public
10 sudo install -m 0644 public/index.html /srv/docview/public/index.html
11 sudo install -m 0644 secret.txt /srv/docview/secret.txt
12 sudo install -d -o docview -g docview -m 0755 /var/log/docviewd /run/
    docviewd
13 sudo install -o docview -g docview -m 0644 /dev/null /var/log/docviewd/
    access.log
14
15 sudo install -m 0644 apparmor/usr.local.bin.docviewd \
16     /etc/apparmor.d/usr.local.bin.docviewd
17 sudo apparmor_parser -r -C -W /etc/apparmor.d/usr.local.bin.docviewd
18
19 sudo -u docview env DOCVIEW_BIND=127.0.0.1 /usr/local/bin/docviewd &
20
21 # legitimate request
22
23 curl -i http://127.0.0.1:8080/
24 curl -i http://127.0.0.1:8080/status
25
26 # post-compromise policy probes
27
28 curl -i http://127.0.0.1:8080/forbidden-read
29 curl -i http://127.0.0.1:8080/forbidden-write
30 curl -i http://127.0.0.1:8080/forbidden-exec
31
32 # inspect AppArmor audit records
```

```

33
34 sudo journalctl -k --since '10 minutes ago' | grep -Ei \
35     'apparmor=.*(ALLOWED|DENIED).*docviewd-demo|docviewd-demo.*apparmor='
36 sudo ausearch -m APPARMOR_ALLOWED,APPARMOR_DENIED,APPARMOR_AUDIT,AVC \
37     -ts recent -i | grep -Ei 'apparmor=|docviewd-demo'
38
39 # switch to production enforcement
40
41 sudo aa-enforce /etc/apparmor.d/usr.local.bin.docviewd
42 sudo pkill -TERM -x docviewd
43 while pgrep -x docviewd >/dev/null; do sleep 0.2; done
44 sudo -u docview env DOCVIEW_BIND=127.0.0.1 /usr/local/bin/docviewd &

```

The `-C` option loads the profile in complain mode, recording accesses outside the allowlist without blocking them. After legitimate and forbidden paths have been requested, `aa-enforce` switches the profile to blocking mode and the daemon is restarted so that the new process enters the enforced domain [47]. In production, the observation phase should be followed by log review with tools such as `aa-logprof`; rules should be added only for legitimate application behavior, not for actions that merely make the probes succeed.

Expected Security Outcome

In complain mode, the forbidden routes may still return successful HTTP responses, but AppArmor records that the operations would have violated the profile. After switching to enforce mode, the same requests fail because the relevant file or execution permissions are absent from the policy. The DAC baseline is important: `/srv/docview/secret.txt` is readable by ordinary users, `/tmp` permits unprivileged file creation, and `/bin/sh` resolves to an executable shell. With those conditions satisfied, the failures are attributable to AppArmor rather than to ordinary Unix permissions.

The expected HTTP behavior follows directly from the C implementation: forbidden reads and writes return 403 `Forbidden` after denied `open()` calls, and forbidden shell execution returns 403 `Forbidden` after AppArmor blocks the shell reached through `system()`. Representative audit records have the following form:

```

1 audit: type=1400 ... apparmor="DENIED" operation="open"
2 profile="docviewd-demo" name="/srv/docview/secret.txt"
3 comm="docviewd" requested_mask="r" denied_mask="r"
4
5 audit: type=1400 ... apparmor="DENIED" operation="open"
6 profile="docviewd-demo" name="/tmp/forbidden.txt"
7 comm="docviewd" requested_mask="w" denied_mask="w"
8

```

```
9 audit: type=1400 ... apparmor="DENIED" operation="exec"  
10 profile="docviewd-demo" name="/usr/bin/dash"  
11 comm="docviewd" requested_mask="x" denied_mask="x"
```

The exact shell path depends on the distribution layout; the security conclusion does not. Since the profile grants no execute transition for the shell, a command-injection path that reaches `system()` cannot continue as an unrestricted helper process. Likewise, the read and write probes fail even after the daemon has accepted the HTTP request and reached the vulnerable operation. AppArmor therefore narrows the post-exploitation action space after the bug is triggered [12], [13], [33].

5.3.10 Implications for Containerized Infrastructure

For the containerized infrastructure studied in this work, AppArmor should be understood as a pragmatic hardening layer for Linux guests running on Proxmox. It does not replace namespaces, cgroups, capability reduction, or seccomp, and it does not remove shared-kernel risk. Its contribution is narrower: it translates intended application behavior into a kernel-enforced profile and denies everything else by default. This explains why AppArmor is attractive on Debian and Ubuntu-oriented deployments and why Docker can build on it through the default `docker-default` profile or an operator-supplied custom profile [12], [13], [47], [50].

5.4 AppArmor in Container Security

5.4.1 Role of AppArmor in Container Isolation

Building on the general MAC semantics established in the previous section, AppArmor becomes operationally relevant when the runtime attaches a profile to container processes. In the Proxmox-hosted Docker architecture examined here, the decisive question is not whether namespaces, cgroups, capabilities, and seccomp exist, but what a compromised process can still do after those mechanisms have defined the container’s basic execution environment.

Namespaces isolate resource views, cgroups constrain consumption, capabilities partition coarse root privileges, seccomp filters system-call entry points, and read-only filesystems reduce persistence opportunities. None of them fully expresses the application-level invariant that a service should read only specific paths, bind only intended socket types, avoid mounting filesystems, avoid tracing neighboring processes, and never spawn helpers outside its expected execution graph.

AppArmor contributes that layer by enforcing per-task policy over files, capabilities, mounts, networking, signals, ptrace, and execution transitions in the kernel itself [3], [4], [12], [13], [47].

This distinction explains why container isolation alone is insufficient as a containment strategy. Docker documents that operators can reduce capabilities with `--cap-drop`, while privileged mode grants all capabilities and reconfigures AppArmor or SELinux to give the container nearly the same access as ordinary host processes. Kubernetes makes the same point by noting that privileged containers override or undo seccomp, AppArmor, and SELinux constraints.

AppArmor therefore does not duplicate seccomp, read-only filesystems, or capability control. It complements them by deciding whether otherwise reachable files, sockets, signals, helper binaries, or mounts are permitted for the confined workload. A successful `openat()` allowed by seccomp may still be denied because the pathname is outside policy; a retained capability may remain unusable if AppArmor does not authorize the corresponding operation; and a writable volume remains safer when policy constrains which subpaths may actually be modified [2], [3], [37], [55], [56].

5.4.2 Default Profiles and Workload-Specific Policy

The practical consequence is that default runtime profiles should be treated as a baseline, not as a workload model. On an AppArmor-enabled host, Docker applies `docker-default` unless the operator overrides it with `--security-opt apparmor=...`. Kubernetes ex-

poses the same design choice through `RuntimeDefault`, `Localhost`, and `Unconfined`; its Pod Security Standards assume `RuntimeDefault` on supported hosts while discouraging or preventing unconfined execution.

Those defaults are useful because they are immediately deployable and materially better than no MAC at all. However, Docker characterizes `docker-default` as “moderately protective” and designed for wide compatibility, so it cannot encode the narrow behavioral invariants of a concrete application. For an Internet-facing reverse proxy, API service, or sidecar³, custom profiles become the mechanism that turns operational assumptions into application-specific least privilege [38], [49]–[51].

5.4.3 Motivating Example: A Hardened Nginx Container

A realistic example is an Nginx container that reverse-proxies requests and serves a static document root from bind-mounted host paths. A GitLab companion implementation provides the AppArmor profile, Nginx configuration, Kubernetes pod manifest, and run script [16]. Docker warns that bind mounts are writable by default and may allow container processes to create, modify, or delete host files. Mounting them read-only is necessary, but it remains a runtime option that can drift under operational pressure, Compose refactoring, or ad hoc troubleshooting.

A custom AppArmor profile adds a second barrier. Even if a bind mount is exposed too broadly, the profile can still deny writes to `/etc/nginx/**` and `/usr/share/nginx/html/**`, deny mount, deny raw sockets, and deny execution of `/bin/sh` or other generic helpers. AppArmor therefore constrains the post-compromise action space of a web-facing process that has already been reached through an application flaw [13], [50], [56].

Implementation Workflow

A sound implementation workflow starts from threat analysis rather than profile syntax. For a containerized service, the profile should identify the startup executable, read-only inputs, writable state directories, required network families, retained capabilities, permitted runtime signals, and any helper programs that may legitimately execute after startup.

The profile should then be drafted from those requirements, loaded initially in complain mode, refined from logs, and switched to enforce mode once legitimate behavior is understood. As described earlier, AppArmor distinguishes complain mode from

³Sidecars, named after sidecars attached to motorcycles, are containerized peripheral components to the services they are deployed alongside [57].

enforce mode, and Docker’s guidance uses the standard load-and-attach cycle through `apparmor_parser`, `aa-status`, and `--security-opt`.

In Kubernetes, the same method is harder because the profile must already be present on the node. The scheduler is not aware of which profiles are loaded where, and Kubernetes requires `Localhost` profiles to be preloaded on suitable nodes. Profile distribution therefore remains a configuration-management concern rather than something expressed solely inside the workload manifest [3], [47], [49]–[51].

Example AppArmor Profile

The following profile illustrates a workload-specific policy for a hardened Nginx web tier. The container uses a static Nginx configuration, runs with a read-only root filesystem, receives writable `tmpfs` mounts only for runtime state, and is started directly with `/usr/sbin/nginx` so that shell-based entrypoints do not have to be permitted. The retained capabilities are limited to binding port 80, preparing the temporary cache directory, and allowing the Nginx master process to drop worker privileges.

```
1 #include <tunables/global>
2
3 profile corp-nginx-web flags=(attach_disconnected,mediate_deleted) {
4     #include <abstractions/base>
5     #include <abstractions/nameservice>
6
7     network inet tcp,
8     network unix stream,
9     deny network raw,
10    deny network packet,
11
12    capability net_bind_service,
13    capability setuid,
14    capability setgid,
15    capability chown,
16
17    /usr/sbin/nginx mr,
18    /usr/sbin/nginx ix,
19
20    /etc/nginx/ r,
21    /etc/nginx/nginx.conf r,
22    /etc/nginx/conf.d/ r,
23    /etc/nginx/conf.d/** r,
24    /usr/share/nginx/html/ r,
25    /usr/share/nginx/html/** r,
```

```

26
27 /run/ rw,
28 /run/nginx.pid rw,
29 /var/run/nginx.pid rw,
30 /var/cache/nginx/ rw,
31 /var/cache/nginx/** rw,
32 /tmp/ rw,
33 /tmp/** rw,
34
35 signal (receive) peer=unconfined,
36
37 deny ptrace,
38 deny mount,
39 deny /proc/** w,
40 deny /proc/sys/** rwklx,
41 deny /proc/sysrq-trigger rwklx,
42 deny /proc/kcore rwklx,
43 deny /sys/** wklx,
44 deny /sys/kernel/security/** rwklx,
45
46 deny /etc/nginx/** w,
47 deny /usr/share/nginx/html/** w,
48
49 deny /bin/** x,
50 deny /sbin/** x,
51 deny /usr/bin/** x,
52 deny /usr/local/bin/** x,
53 }

```

Listing 5.2: Custom AppArmor profile for a hardened Nginx container.

The security value of this profile lies in the way it turns deployment assumptions into kernel-enforced invariants. `network inet tcp` and `network unix stream` allow the sockets Nginx needs, while `deny network raw` and `deny network packet` remove raw and packet socket access. The retained capabilities are limited to binding port 80, preparing temporary cache state, and allowing the master process to drop worker privileges.

The rules for `/etc/nginx/**` and `/usr/share/nginx/html/**` separate read-only inputs from writable state, so a compromised worker cannot overwrite configuration, implant a persistent web shell in the served document tree, or tamper with static content even if an operator accidentally changes a bind mount from `ro` to `rw`. `deny mount` and the `/proc` and `/sys` denials block kernel-facing operations that are not legitimate for a web tier.

Finally, execution is granted only for `/usr/sbin/nginx` via `ix`, which keeps permitted

re-execution within the same profile. Because there is no execute permission for `/bin/sh`, `/usr/bin/env`, or other generic helpers, command-injection follow-up steps that try to spawn a shell are denied at the MAC layer [13], [37], [47], [56].

Attaching the Profile with Docker

The profile is attached to a container through the runtime. A hardened Docker deployment can therefore combine runtime controls and AppArmor as follows:

```
1 sudo apparmor_parser -r -W /etc/apparmor.d/containers/corp-nginx-web
2
3 docker run -d --name edge-nginx \
4     --pull=missing \
5     --entrypoint /usr/sbin/nginx \
6     --read-only \
7     --tmpfs /run:rw,noexec,nosuid,nodev,size=16m \
8     --tmpfs /var/cache/nginx:rw,noexec,nosuid,nodev,size=64m,mode=1777 \
9     --mount type=bind,src=/srv/nginx/nginx.conf,dst=/etc/nginx/nginx.conf,
10    ro \
11    --mount type=bind,src=/srv/nginx/conf.d,dst=/etc/nginx/conf.d,ro \
12    --mount type=bind,src=/srv/nginx/html,dst=/usr/share/nginx/html,ro \
13    --cap-drop ALL \
14    --cap-add NET_BIND_SERVICE \
15    --cap-add SETUID \
16    --cap-add SETGID \
17    --cap-add CHOWN \
18    --security-opt apparmor=corp-nginx-web \
19    --security-opt no-new-privileges=true \
20    -p 8080:80 \
    nginx:1.27-alpine -g 'daemon off;'
```

This command makes AppArmor effective because it is not used in isolation. `--read-only` prevents writes to the image filesystem; `--tmpfs` creates narrow writable islands for PID and cache state; the bind mounts are read-only; `--cap-drop ALL` followed by a narrow `--cap-add` set keeps only required capabilities; and `no-new-privileges` prevents later privilege gain through an execution boundary.

AppArmor then governs what remains reachable inside that reduced environment. If the service is compromised and attempts to write into `/usr/share/nginx/html`, run `/bin/sh`, mount a host path, or alter `/proc/sys`, the kernel denies the operation even after the attack has reached code execution inside the container [2], [37], [50], [55], [56].

5.4.4 Operational Limitations

Operationally, the main limitations are administrative. AppArmor policy quality is path-dependent: image updates, altered entrypoints, added helper binaries, or changed mount layouts can invalidate a previously correct profile or tempt operators into broad globs that preserve availability at the expense of confinement.

Debugging under strict confinement can also be frictional because the profile denies common troubleshooting habits such as `docker exec ... /bin/sh`, `strace`, or ad hoc package installation.

Finally, AppArmor provides little value if operations frequently fall back to `Unconfined` or `--privileged`. Docker warns that privileged containers are not securely sandboxed. The practical risk is therefore that policy is relaxed until it becomes semantically equivalent to the generic default profile, discarding the workload-specific value of the mechanism [3], [37], [49]–[51].

5.4.5 Summary and Transition to SELinux

Within the broader argument of this paper, AppArmor should therefore be understood as the mechanism that translates least-privilege assumptions for containerized services into enforceable runtime policy on AppArmor-capable hosts. It does not eliminate kernel-level risk, and it does not replace the namespace, cgroup, capability, seccomp, or read-only filesystem measures discussed elsewhere in the paper.

Its value is narrower and more tangible: it limits which paths, socket classes, executable transitions, and kernel-facing interfaces remain available after a container process has already been reached by an attacker. This practical, path-oriented approach also clarifies the transition to the following SELinux section: the relevant comparison is not whether MAC is needed for container security, but how different MAC models express, deploy, and maintain that confinement in production [3], [4], [12], [13].

5.5 Security-Enhanced Linux (SELinux)

5.5.1 What is SELinux?

SELinux (Security-Enhanced Linux) is a Linux Security Module that implements MAC through labels and type-enforcement policy. It was originally developed by the NSA, integrated into the Linux kernel in 2003, and is now strongly associated with Red Hat-family systems, Android, and other distributions that enable or support SELinux policy enforcement [58].

Its security model follows least privilege through default denial: an operation is allowed only when the active policy explicitly permits the subject's label to access the target object's label. This applies even to privileged processes, so root authority alone is not sufficient to bypass SELinux confinement [14].

5.5.2 How Does It Work?

When a subject, usually a process, attempts to access an object such as a file, port, or socket, SELinux checks whether the active policy permits that subject context to perform the requested operation on the target context. Previously resolved decisions are cached in the Access Vector Cache (AVC); unresolved requests are evaluated by the SELinux security server against the loaded policy database. Denials are recorded as AVC messages, usually in `/var/log/audit/audit.log` when `auditd` is available.

The most common policy model on general-purpose systems is the targeted policy, which confines selected system services while leaving many interactive user sessions unconfined. More restrictive configurations, including MLS, can confine users and processes more broadly. SELinux can operate in three modes:

- Enforcing: policy decisions are applied; denied operations are blocked and logged;
- Permissive: policy violations are logged but not blocked, which is useful during diagnosis and policy development;
- Disabled: no SELinux policy is loaded or enforced. This mode is discouraged because files created without SELinux labels may require relabelling before enforcement can be safely restored.

Labelling

To enforce policy, SELinux assigns processes, users, files, network ports, and other objects a security context, also known as a security label. The standard context format is `user:role:type:level:`

- User: the SELinux identity mapped from a Linux account;
- Role: the authorized SELinux role, commonly ending in `_r`;
- Type: the main enforcement field, commonly ending in `_t`, such as `httpd_t`, `bin_t`, or `postgresql_port_t` [52];
- Level: the optional MLS or MCS field, such as `s0` or `s0:c1,c2`.

For processes, the type field is usually called a domain; for objects, it remains the object type. Type Enforcement (TE) is driven primarily by this field: SELinux compares the subject domain with the target type and object class to decide whether actions such as read, write, execute, or bind are permitted [59].

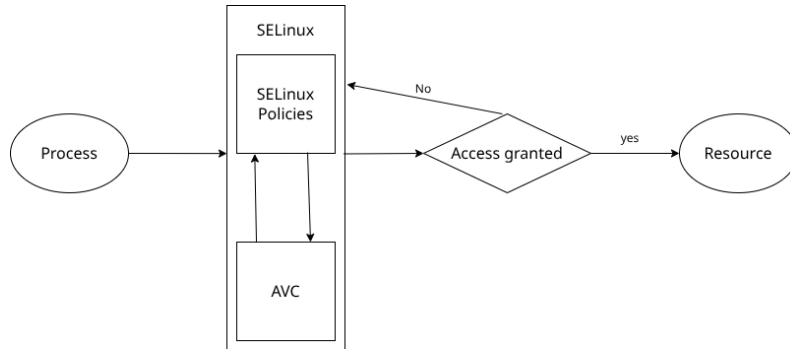


Figure 5.4: Basic operating principle of SELinux.

SELinux use cases

Organizations use SELinux mainly where policy-enforced containment is required:

- Server hardening: a compromised service remains limited by its domain even when it runs with elevated Unix privileges;
- Compliance requirements: regulated environments often require centrally enforced MAC controls;
- Zero-trust architectures: processes can access only resources explicitly allowed by policy;
- Container security: container platforms can use SELinux labels and categories to isolate workloads.

Users and Roles (RBAC)

SELinux maps Linux users (like root) to SELinux users (like `system_u` or `unconfined_u`). In a default targeted policy, most accounts, including root, are mapped to `unconfined_u` and run in the `unconfined_t` domain. Processes in this domain are not subject to Type Enforcement restrictions, which preserves day-to-day usability for interactive sessions while still confining the system daemons that present the highest risk.

Processes started by systemd can transition into confined domains such as `httpd_t` or `sshd_t`. This is the defining characteristic of the targeted policy: high-risk system dae-

mons are confined, while many interactive sessions remain unconfined. Stricter policies, including MLS-oriented deployments, can confine users more comprehensively.

Level (MLS/MCS) and Containers

The level field supports MLS and, in container environments, Multi-Category Security (MCS). A container process may receive a category pair such as `s0:c1,c2`, while another receives `s0:c3,c4`. SELinux policy can then require the process and its files to carry matching categories, preventing one container domain from reading or writing another container's filesystem even when both use the same base type. The Docker-specific use of this mechanism is discussed in the following SELinux container section.

5.5.3 Practical Introduction to SELinux

This section introduces some fundamental commands that provide a practical grounding in SELinux administration [60].

Booleans

Booleans are policy switches that enable or disable optional rules without editing policy source. They are inspected with `getsebool` and changed with `setsebool`; the `-P` flag makes the change persistent:

```
1 getsebool -a
2 setsebool -P httpd_can_network_connect on
```

`httpd_can_network_connect` is a common boolean that controls whether the web server (`httpd_t`) can establish outbound network connections.

Policy Rules

SELinux policy rules use the following basic syntax:

```
<rule_type> <subject> <object>:<object_class> <permission>;
```

The rule type is usually `allow`, `dontaudit`, or `neverallow`; the subject is the requesting domain, the object is the target type, the class identifies the kind of object, and the permission names the operation. The following rule permits a process labelled `apache_process` to read a file labelled `apache_log`:

```
1 allow apache_process apache_log:file read;
```

Before writing new rules, administrators should check whether an existing boolean or policy rule already covers the intended behavior. The later `sesearch` and `seinfo` examples address that workflow.

Changing SELinux Mode

`getenforce` returns the current SELinux mode, while `sestatus` reports mode, policy, and configuration details. `setenforce` changes the runtime mode: 0 switches to permissive mode and 1 restores enforcing mode.

```
1 getenforce
2 sestatus
3 setenforce MODE_ID
```

This change does not survive a reboot. To make it persistent, the SELinux configuration file at `/etc/selinux/config` must be edited. This file is also the only mechanism for setting `SELINUX=disabled` and all the changes to it take effect after the next reboot.

Fixing contexts

`chcon` changes a file's SELinux context directly in the `security.selinux` extended attribute, but it does not update the policy database. The change can therefore be overwritten by any later relabelling operation, including `restorecon`. The following example assigns the type `admin_home_t` to `/root/xfile`:

```
1 chcon -t admin_home_t /root/xfile
```

The preferred alternative to `chcon` is `restorecon`:

```
1 #-v flag is for verbosity, -R stands for "recursive"
2 restorecon -vR /var/www/html/new_website
```

This command reapplies the default context from the policy database rather than leaving a one-off extended-attribute override.

If a path has no matching file-context rule, `restorecon` will leave it unchanged. A persistent rule can be added with `semanage fcontext`; this example assigns `httpd_sys_script_exec_t` to a custom web application path:

```
1 semanage fcontext -a -t httpd_sys_script_exec_t "/opt/custom_web_app/(.*)?"
   "
2 restorecon -vR /opt/custom_web_app
```

Troubleshooting

Most SELinux issues originate from incorrect file labelling, particularly when files are moved between directories or when custom applications are deployed outside the paths covered by the default policy. When diagnosing such problems, disabling SELinux is strongly discouraged: switching to permissive mode achieves the same diagnostic goal while keeping the logging system active.

The `-Z` flag, available on commands such as `ls` and `ps`, shows SELinux contexts. The most useful first commands are:

```
1 sestatus
2 ls -lZ /var/www/html
3 ps -eZ | grep httpd
4 ausearch -m AVC
```

When transitioning from disabled mode to enforcing mode, a full relabel is often required. The recommended approach is to schedule it for the next boot rather than relabelling a live system:

```
1 touch /etc/selinux/.autorelabel
2 reboot
```

Copying and moving files also affects labels. `cp` makes the new file inherit the destination directory's context. `mv` preserves the original context only when source and destination are on the same filesystem; across filesystems it behaves like copy-and-delete and the new file inherits the destination context.

Auditing logs

Audit logs are central to the SELinux troubleshooting workflow. The file `/var/log/audit/audit.log` contains the raw AVC denial records but its format can be difficult to interpret directly. The `setroubleshoot-server` package provides `sealert`, which parses the raw log and produces a human-readable report with suggested remediation steps.

```
1 sealert -a /var/log/audit/audit.log
```

If `sealert` is insufficient, `audit2why` can explain why a denial occurred:

```
1 ausearch -m AVC -ts recent | audit2why
```

When the denial is legitimate and no existing boolean or context correction solves it, `audit2allow` can generate policy source or a loadable module:

```
1 ausearch -m AVC -ts recent | audit2allow -m myCustomAppPolicy
2 ausearch -m AVC -ts recent | audit2allow -M myCustomAppPolicy
3 semodule -i myCustomAppPolicy.pp
```

`-m` emits a human-readable `.te` file for inspection; `-M` creates both the `.te` file and a compiled `.pp` package. The module is not active until it is installed with `semodule -i`. Generated rules should be reviewed before installation, because they permit whatever denial was observed, not necessarily what the administrator intended.

Limitations of audit2allow

`audit2allow` is mechanical: it reads denial entries and produces rules that would suppress them. This creates two risks. First, a single denied operation may be only one step in a broader access path, so allowing it may simply expose the next denial. Second, the tool has no knowledge of administrator intent; a rule generated from malicious or buggy behavior can accidentally legitimize that behavior.

Managing Ports and Users with semanage

Port labelling. Each network port carries a SELinux type label, and a process may bind to or connect on a port only if its domain is allowed to use that port type. The default policy covers common assignments, but non-standard service ports require manual labelling. This example registers TCP port 8080 as `http_port_t`:

```
1 # Add a custom port to an existing type
2 semanage port -a -t http_port_t -p tcp 8080
3
4 # List all port-to-type mappings currently defined
5 semanage port -l
```

Absent this entry, a web server reconfigured to listen on port 8080 will be denied by SELinux at bind time.

User and login mappings. On a default targeted system, most Linux accounts are mapped to `unconfined_u`. Where stricter confinement is required, a Linux user can be associated with a confined SELinux user through the `login` subcommand:

```
1 # Map a Linux user to a confined SELinux user
2 semanage login -a -s user_u <user_username>
3
4 # Display the current login mappings
5 semanage login -l
```

Once mapped to `user_u`, the account is confined to the `user_t` domain and cannot perform role transitions toward privileged domains such as `sysadm_t`, regardless of what the traditional Unix permission model would otherwise allow.

Interrogating the Policy with sesearch and seinfo

A common SELinux troubleshooting mistake is reacting only after denials occur. The `seinfo` and `sesearch` utilities, part of the `setools` package, query the active policy directly and show what a domain is already allowed to do.

`seinfo` reports structural information such as types, roles, users, classes, and Boolean-controlled rules:

```
1 # Count the types, roles and allow rules in the active policy
2 seinfo
3
4 # List all types whose name contains "httpd"
5 seinfo -t | grep httpd
```

`sesearch` performs rule-level searches and helps determine whether a specific allow rule already exists:

```
1 # Show all allow rules where httpd_t is the source domain
2 sesearch --allow -s httpd_t
3
4 # Narrow the search to rules involving a specific target type
5 sesearch --allow -s httpd_t -t var_t
6
7 # Show rules whose evaluation is conditioned on a specific Boolean
8 sesearch --allow -s httpd_t -b httpd_can_network_connect
```

Used together, `seinfo` and `sesearch` help avoid redundant custom modules before resorting to `audit2allow`.

Domain Transitions

Type Enforcement governs not only access to static resources but also the domain a process occupies after invoking `execve()`. When a confined process executes a binary whose file type is marked as an entry point for a different domain, the kernel switches the child process into that new domain rather than inheriting the parent's one. This mechanism is known as a *domain transition*.

A valid transition requires four policy conditions: the source domain must have `execute` permission on the binary, the target domain must have `entrypoint` permission on that binary, the source must have `transition` permission toward the target domain, and an explicit `type_transition` rule must tie the relationship together.

```
1 # The four rules that together authorise a domain transition
2 allow source_domain_t entry_binary_t : file      execute;
3 allow source_domain_t target_domain_t : process transition;
4 allow target_domain_t entry_binary_t : file      entrypoint;
5 type_transition source_domain_t entry_binary_t : process target_domain_t;
```

This explains why system daemons can enter their own confined domains despite being launched by `systemd_t`. For example, `/usr/sbin/httpd` carries the type `httpd_exec_t`,

which is declared as an entry point for `httpd_t`; the transition rule places the new process in `httpd_t`, where only the web-server policy applies.

An entry point type is necessary but not sufficient: a user script that calls the same binary will not enter `httpd_t` unless the user's domain also appears in a matching `type_transition` rule.

File Context Equivalency

When a service's data directory is moved to a non-standard path, writing a new `fcontext` rule with an explicit type works but can become fragile: upstream updates to the canonical path rules will not automatically apply to the custom entry.

A more robust approach is to declare the new path as *equivalent* to the canonical one. The `semanage fcontext` subcommand supports this through the `-e` flag:

```
1 # Declare /data/postgres as equivalent to /var/lib/pgsql
2 semanage fcontext -a -e /var/lib/pgsql /data/postgres
3
4 # Apply the inherited context rules to the new path
5 restorecon -vR /data/postgres
```

With this entry in place, `restorecon` treats `/data/postgres` as a structural mirror of `/var/lib/pgsql`. Future updates to the canonical rules are inherited automatically.

Managing Policy Modules with `semodule`

The `semodule` command manages policy modules loaded into the running kernel. Besides installing a compiled `.pp` package with `-i`, common operations include:

```
1 # List all modules currently loaded, with name and version
2 semodule -l
3
4 # Remove a previously installed module
5 semodule -r myCustomAppPolicy
6
7 # Disable a module without removing it (available in newer versions)
8 semodule --disable myCustomAppPolicy
```

The `-r` flag unloads the module and rebuilds the active policy immediately, without requiring a reboot.

The `dontaudit` Rule Type

Beyond `allow` and `neverallow`, `dontaudit` denies the specified access while suppressing the corresponding AVC message. It is used to silence harmless expected denials, but it

can hide useful troubleshooting evidence when a service fails silently. Policy query tools can expose these rules:

```
1 # Search specifically for dontaudit rules involving a given domain
2 sestatus --dontaudit -s my_app_t
```

`dontaudit` rules can also be disabled temporarily so suppressed denials appear in the audit log:

```
1 # Disable all dontaudit rules (useful during deep troubleshooting)
2 semodule -DB
3
4 # Re-enable them once the investigation is complete
5 semodule -B
```

5.6 SELinux in Docker Containers

Docker integrates with SELinux through the kernel's LSM infrastructure. When SELinux is enabled on the host and Docker is started with SELinux support active, container processes and files receive security contexts that the kernel enforces like ordinary SELinux labels. The difference is that Docker assigns container-specific domains, file types, and MCS categories so that container workloads remain separated from host resources and from each other.

5.6.1 How Docker Uses SELinux

On systems such as Rocky Linux, Docker may run without SELinux support unless it is explicitly enabled in the daemon configuration. In that case, containers can run with the `spc_t` type, an unconfined SELinux domain sometimes described as a super-privileged container domain. This is different from Docker's `--privileged` flag, which also changes capabilities, seccomp, devices, and other runtime boundaries.

With the `container-selinux` policy installed, the Docker daemon and low-level runtime binaries carry the `container_runtime_t` domain. When SELinux support is active, container processes run as `container_t`. The transition does not rely on the ordinary entrypoint mechanism described earlier; instead, the runtime uses `setexeccon()` to assign the desired label to the next `execve()` call. The policy permits this through a dynamic transition rule:

```
1 # Specific policy which implements this behaviour
2 allow container_runtime_domain container_domain:process { dyntransition
   transition };
```

This design is necessary because any executable type can be the first process inside a container; declaring every possible binary as an entrypoint for `container_t` would be impractical.

Files in the container's layered filesystem receive the type `container_file_t`. The policy grants `container_t` broad access to that type but withholds access to host-side types such as `user_home_t` or `etc_t`. A process that escapes a container namespace therefore still carries the `container_t` domain and remains unable to access host files whose types are outside policy.

In addition to the type label, each container receives a unique MCS category pair, for example `s0:c42,c317`. The policy allows a container process to access only files with the same category pair, so two processes running as `container_t` remain separated when their `container_file_t` files carry different categories.

The assigned labels can be inspected with `ps -Z`:

```
1 ps -efZ
2 # Example output from nginx and sleeping alpine containers:
3 system_u:system_r:container_t:s0:c260,c643 root 3159 3135 0 09:30 ?
4     00:00:00 nginx: master process nginx -g daemon off;
5 ...
6 system_u:system_r:container_t:s0:c556,c801 root 3387 3364 0 09:33 ?
7     00:00:00 sleep infinity
```

5.6.2 Enabling SELinux Support in Docker

To enable SELinux enforcement for Docker containers persistently, set `selinux-enabled` in the daemon configuration and restart Docker:

```
1 # /etc/docker/daemon.json
2 {
3     "selinux-enabled": true
4 }
5
6 systemctl restart docker
```

The current state can be inspected with:

```
1 docker info | grep -i selinux
2 # Output if enabled:
3 Security Options:
4     seccomp
5     Profile: builtin
6     selinux
7     cgroupns
```

SELinux can also be controlled per container through `--security-opt`:

```
1 # Run a container with a custom type (the type has to be defined in a
   policy)
2 docker run --security-opt label=type:my_custom_container_t myimage
3
4 # Disable SELinux labelling for a single container (not recommended in
   production)
5 docker run --security-opt label=disable myimage
```

Disabling the label removes the SELinux boundary for that workload and places it outside the intended `container_t` confinement model.

5.6.3 Bind Mounts and Volume Labelling

The most common source of SELinux-related container failures is the bind mount. When a host directory is mounted into a container with `-v` or `--volume`, its files retain their original SELinux type, such as `user_home_t` or `var_t`. Since `container_t` is not allowed to access those types, the container may start normally while the application cannot read or write the mounted directory.

The denial appears in the host audit log:

```
1 ausearch -m AVC | grep container_t
2 # Example denial:
3 type=AVC msg=audit(...): avc: denied { read } for pid=4102 comm="nginx"
4   path="/var/data/app" scontext=system_u:system_r:container_t:s0:c5,c200
5   tcontext=unconfined_u:object_r:var_t:s0 tclass=dir permissive=0
```

Docker provides two suffixes that relabel the host directory before mounting it:

- `:z`: shared label. The directory is relabelled as `container_file_t` with a shared category so multiple containers can access it;
- `:Z`: private label. The directory is relabelled as `container_file_t` with the specific MCS category pair of the container being started, so only that container can access it.

```
1 # Shared relabelling (accessible to all containers)
2 docker run -v /var/data/app:/app:z myimage
3
4 # Private relabelling (accessible only to this container)
5 docker run -v /var/data/app:/app:Z myimage
```

The relabelled context is visible on the host:

```

1 # Before mounting
2 ls -Z /var/data/app
3 unconfined_u:object_r:var_t:s0 app
4
5 # After mounting with :Z
6 ls -Z /var/data/app
7 system_u:object_r:container_file_t:s0:c5,c200 app

```

The `:Z` suffix modifies the directory's extended attributes in place. If the directory is later mounted into another container, the stored MCS category may not match the new container's category pair and access will be denied. In that case, `restorecon` can restore the original context before remounting.

5.6.4 Writing Custom Policies for Containers

When a containerized application requires permissions that `container_t` does not hold by default, the general audit workflow still applies: collect AVC denials, generate candidate rules, and review the resulting `.te` file before loading it. This review is especially important for containers because a busy workload can generate many denials and `audit2allow` will attempt to permit all observed behavior.

A more purpose-built alternative is `udica`, which generates a tailored SELinux policy from container inspection data. Unlike `audit2allow`, which reacts to denials, `udica` reads declared volumes, exposed ports, and Linux capabilities from `docker inspect` and builds a policy for that container shape.

```

1 # Export the container's inspection data
2 docker inspect mycontainer > mycontainer.json
3
4 # Generate a policy based on declared mounts, ports and capabilities.
5 # udica prints the exact semodule command to run as part of its output.
6 udica -j mycontainer.json myContainerPolicy

```

After generation, the policy modules printed by `udica` must be loaded and the container restarted with the generated type:

```

1 # An example of the output of the command "udica -j mycontainer.json
   myContainerPolicy"
2
3 Policy myContainerPolicy created!
4
5 Please load these modules using:
6 # semodule -i myContainerPolicy.cil /usr/share/udica/templates/
   base_container.cil

```

```
7
8 Restart the container with: "--security-opt label=type:myContainerPolicy.
   process" parameter
```

Unlike `audit2allow`, `udica` produces a human-readable CIL file rather than a `.pp` binary package. Both formats can be loaded with `semodule`, but the CIL source remains easier to inspect and adjust before loading.

5.6.5 The `--privileged` Flag and Its Implications

Several runtime options can remove SELinux confinement for a container:

- `--privileged`;
- `--pid=host` and `--ipc=host`;
- `--security-opt label=disable`.

In these cases, the process may be assigned the `spc_t` domain or otherwise bypass the normal `container_t` confinement path.

A process in `spc_t` can access host resources much like an unconfined process. `--privileged` should therefore be treated as an exceptional administrative escape hatch, not as a solution for permission problems that should instead be resolved with a targeted boolean, a corrected file context, or a custom policy module.

Chapter 6

Attack Scenarios and Countermeasures

6.1 Chapter Overview

This chapter focuses on a containerized infrastructure deployed on Proxmox VE, where Docker workloads are protected by the Linux isolation mechanisms discussed earlier and by AppArmor or SELinux policy enforcement. For each scenario, the discussion identifies the attacker’s objective, the required foothold, the repository-backed execution path, the expected impact on protected assets, the detection indicators and the defensive controls that reduce likelihood or impact.

The chapter therefore connects the architectural and threat-model discussion with an operational assessment of the protection mechanisms. The analysis is qualitative: it does not claim measured exploit probabilities, but it uses a consistent likelihood-impact model to compare initial and residual risk across scenarios.

6.2 Methodology and Scenario Setup

6.2.1 Infrastructure Under Analysis

The infrastructure under analysis consists of four layers, each with its own enforcement boundary and residual risk:

1. **Proxmox VE node:** the management plane, QEMU processes, storage paths and cluster configuration are trust-critical. The analysis does not assume that Proxmox-provided AppArmor or SELinux policy alone confines these services; vendor patch-

ing, API permissions, network restriction and local hardening remain primary controls.

2. **AppArmor VM:** an Ubuntu/Debian-family VM where AppArmor is the active Linux Security Module (LSM). Docker containers run with `docker-default` or with workload-specific profiles that restrict filesystem writes, executable transitions, and selected kernel interfaces.
3. **SELinux VM:** a RHEL-family guest where SELinux runs in enforcing mode with the targeted policy. The Docker SELinux discussion uses container domains and MCS category pairs, while the ABRT scenario uses the Fedora 42 guest described in the GitLab companion repository.
4. **Container layer:** Docker namespaces, cgroups, capabilities, seccomp, AppArmor, and SELinux labels combined to isolate container processes from the host and from each other.

The scenario implementations used to support the chapter are published in the public GitLab companion repository `cns-codeexamples` [16].

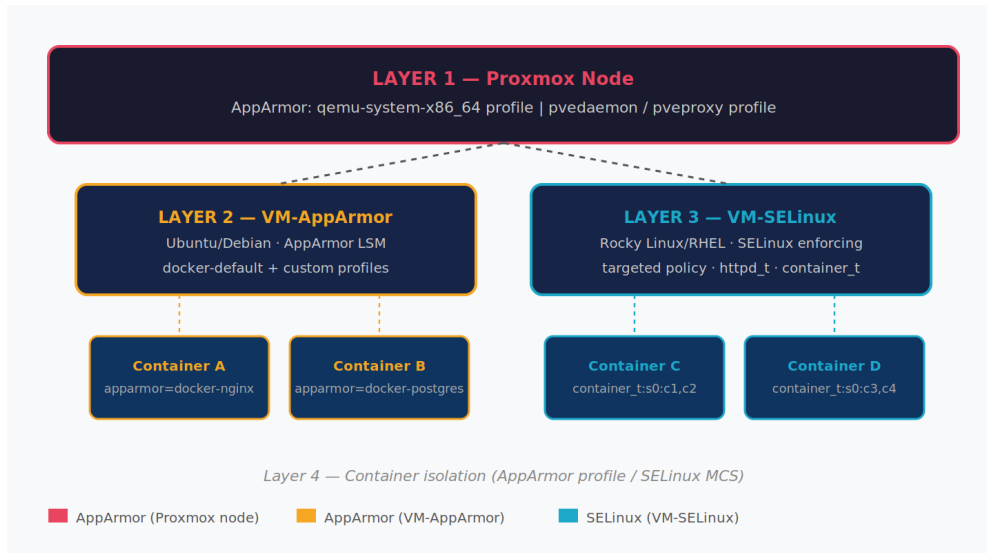


Figure 6.1: Four-layer stack used in the scenario analysis: Proxmox node, guest VM, LSM-enforced runtime, and container workload.

6.2.2 Scenario Construction Criteria

The scenarios are constructed according to the following criteria:

- **Threat actor assumptions:** the adversary is assumed to have a realistic initial foothold, such as code execution inside a VM, root inside a container, access to an internal network segment or possession of a weak credential. The model does not assume physical access, complete prior knowledge of the infrastructure or unlimited capability.
- **Protected assets:** the critical assets are the Proxmox node, cluster authentication material such as `/etc/pve/priv/authkey.key`, guest VM disks and configuration, Docker workloads, application data and service availability.
- **Attack surfaces:** the relevant surfaces are QEMU/KVM device emulation, VirtIO and `virtiofs`, the Docker daemon and API socket, Linux kernel interfaces exposed to containers, shared filesystems, exposed management ports and misconfigured MAC profiles.
- **Trust boundaries:** the primary boundaries are between guest and host, between container and VM, between container namespaces and host resources and between the internal workload network and the Proxmox management network.
- **Preconditions:** each scenario states the minimum conditions required for the attack, such as root inside a container, a mounted Docker socket, a reachable Proxmox API or a disabled enforcement profile.

6.2.3 Risk Evaluation Model

Risk is evaluated qualitatively by combining likelihood and impact:

$$\text{Risk} = \text{Likelihood} \times \text{Impact} \quad (6.1)$$

Likelihood expresses how feasible the attack is under the stated preconditions, considering exploit availability, configuration exposure and required skill. Impact expresses the expected technical and operational consequences, including loss of confidentiality, integrity, availability, authentication, authorization and accountability. The final severity is represented using qualitative levels such as low, moderate, high and critical, following common risk-matrix practice [61], [62].

Residual risk is then estimated after applying the proposed controls. The residual values should be read as comparative judgements rather than measured probabilities: they indicate whether the countermeasures reduce the attack surface, contain the blast radius, improve detection or leave a significant operational exposure.

6.3 Scenario 1: `runc` Container Escape via CVE-2024-21626 Contained by AppArmor

6.3.1 Scenario Description

This scenario analyzes a container runtime escape based on CVE-2024-21626, a vulnerability in `runc` that affected the container process creation and execution path before the corrected release. The vulnerability is relevant to Docker-based infrastructures because Docker relies on lower-level OCI runtimes such as `runc` to create the final container process. If that runtime mishandles file descriptors and the process working directory, a container process can be started with a working directory that resolves through `/proc/self/fd/N` and later through `/proc/self/cwd`, allowing access to filesystem locations outside the intended container root [63], [64].

The GitLab companion scenario compares three cases: vulnerable `runc v1.1.11` without AppArmor, the same vulnerable `runc` binary with a custom AppArmor profile, and fixed `runc v1.1.12` without AppArmor. AppArmor does not remove the CVE from the runtime; it hardens the execution environment so that the escaped process cannot use `/proc/self/cwd` traversal to create the host marker. The baseline escape succeeds, while the AppArmor-constrained run blocks the host write before patching is considered.

MITRE ATT&CK classifies this type of activity as Escape to Host (T1611). Recommended mitigations include runtime patching, restricted privileges, seccomp, read-only filesystems, and avoidance of privileged containers [65].

6.3.2 Threat Model and Preconditions

- **Attacker capability:** the attacker can influence container process execution, for example through command execution in a compromised workload, a malicious image, or an exposed container execution path.
- **Required access:** the attacker does not initially control the VM host, the Proxmox node, or the Docker daemon configuration. The companion script invokes pinned `runc` binaries directly to reproduce the runtime condition without changing the system runtime.
- **Target components:** vulnerable `runc v1.1.11`, the OCI bundle, leaked file descriptors, `/proc/self/fd/N`, `/proc/self/cwd`, the container root filesystem, and the AppArmor profile applied to the executed process.

- **Assumptions:** the host is AppArmor-enabled, the scenario container uses a minimal BusyBox root filesystem, `no-new-privileges` is enabled, the capability sets are emptied, and the only intentional difference between the vulnerable baseline and the hardened run is AppArmor confinement.
- **Assets at risk:** host filesystem integrity, host paths reachable through the leaked descriptor, runtime state, workload data, and any secrets exposed to the container process.

6.3.3 Practical Implementation

The GitLab companion repository contains an implementation of this comparison. It downloads the official `runc` release binaries for the target architecture, uses the upstream checksum material, builds a small OCI bundle from a static BusyBox root filesystem, and runs the same escape probe under three configurations. The default mode deliberately opens a read-only host-root file descriptor before executing vulnerable `runc`. This explicit descriptor makes the result deterministic while still exercising the vulnerable `runc exec --cwd /proc/self/fd/N` path.

```
1 # Repository comparison from the GitLab companion repository.
2 sudo -v
3 ./run-demo.sh compare
```

The payload is intentionally narrow. It does not modify system files, install persistence, change Docker's configured runtime, or scan the network. It only attempts to create a marker named `/tmp/runc-cve-2024-21626-*.marker` on the host by using `/proc/self/cwd` traversal after the process has been started with the crafted working directory.

```
1 # Expected comparison result.
2 == Baseline: vulnerable runc without AppArmor ==
3   runc:      runc version 1.1.11
4   exec cwd:  /proc/self/fd/9
5   profile:   unconfined
6   marker:    yes
7
8 == Hardened: vulnerable runc with AppArmor confinement ==
9   runc:      runc version 1.1.11
10  exec cwd:   /proc/self/fd/9
11  profile:    runc-cve-2024-21626-lab
12  marker:     no
13  denial:     Permission denied on /proc/self/cwd/...marker
14
```

```

15 == Fixed: runc v1.1.12 without AppArmor ==
16   runc:      runc version 1.1.12
17   exec cwd:  /proc/self/fd/9
18   profile:   unconfined
19   marker:    no
20   output:    chdir to cwd "/proc/self/fd/9" failed

```

The AppArmor profile used for the hardened run is workload-specific. It permits the BusyBox workload to start and read the minimal files it needs, but denies broad post-exploitation behavior: `ptrace`, `mount`, network access, all capabilities, writes through `/proc/**`, traversal through `/proc/self/cwd/**` and `/proc/**/cwd/**`, and writes to common host paths such as `/tmp/**`, `/etc/**`, `/usr/**`, `/root/**`, and `/home/**`. In the context of this scenario, the profile is the hardening control that turns a reachable sandbox escape primitive into a blocked host write [12], [13], [50].

6.3.4 Expected Impact

Without AppArmor confinement and without the fixed runtime, the vulnerable execution path can create a file on the host from a process that should have remained inside the container root filesystem. The repository workflow writes only a harmless marker under `/tmp`, but the security meaning is broader: the container boundary has failed, and host paths reachable through the leaked descriptor may become targets for tampering, staging, persistence, or sensitive-data access depending on permissions and deployment context.

With the custom AppArmor profile enforced, the same vulnerable `runc` version is still used and the same CVE-oriented probe is attempted. The difference is that kernel policy denies the filesystem actions needed to turn the sandbox escape into a host modification. The residual impact is therefore reduced from host-level write capability to a blocked exploitation attempt with auditable AppArmor denials. The fixed `runc` case provides the proper remediation by removing the vulnerable behavior from the runtime itself.

6.3.5 Detection Indicators

- **Runtime command indicators:** suspicious `runc exec` or equivalent container execution paths using `--cwd /proc/self/fd/N`, unexpected working directories under `/proc/self/fd/`, or direct invocation of non-system `runc` binaries.
- **Host filesystem indicators:** unexpected files matching `/tmp/runc-cve-2024-21626-*.marker`, or comparable host-side files created by a process associated with container execution.

- **AppArmor denials:** kernel records containing `apparmor="DENIED"` for profile `runc-cve-2024-21626-lab`, especially denials involving `/proc/self/cwd/**`, `/proc/**/cwd/**`, `/tmp/**`, `mount`, `ptrace`, capabilities, or network use.
- **Version inventory:** hosts still running vulnerable `runc` releases, especially `v1.1.11` in the repository comparison, should be treated as exposed until updated.
- **Container configuration drift:** privileged containers, disabled AppArmor profiles, excessive capabilities, host namespace sharing, and writable host bind mounts increase the practical impact of any runtime escape path.

6.3.6 Countermeasures

- **Patch the runtime:** update `runc` to `v1.1.12` or later. AppArmor containment reduces exploit impact, but the vulnerability is fixed by replacing the affected runtime.
- **Apply AppArmor hardening:** attach a profile that denies `/proc/self/cwd/**` traversal, writes through `/proc/**`, writes to common host paths, mount operations, `ptrace`, network use, and unnecessary capabilities while permitting only the workload's expected files and executables [12], [13], [50].
- **Preserve Docker least privilege:** avoid `--privileged`, avoid host bind mounts and `/var/run/docker.sock`, drop unnecessary capabilities, and enforce `no-new-privileges` [4], [37].
- **Reduce syscall exposure:** keep the Docker seccomp profile enabled and avoid broad exceptions for high-risk operations unless they are explicitly required by the workload [2].
- **Monitor runtime and LSM state:** alert on disabled AppArmor profiles, containers launched as `unconfined`, vulnerable `runc` versions, and kernel denials associated with container execution.

6.3.7 Risk Analysis

Table 6.1: Qualitative risk assessment for Scenario 1.

Factor	Before countermeasures	Initial value	After countermeasures	Residual value
Likelihood	Vulnerable and <code>/proc/self/fd/N</code> execution path reachable <code>runc</code>	Medium/High	Runtime update, AppArmor profile, no-new-privileges, seccomp	Low
Impact	Host filesystem write through container escape primitive	High	Host write blocked by AppArmor; vulnerability removed by fixed <code>runc</code>	Low/Moderate
Overall risk	Container sandbox escape becomes host integrity compromise	High	CVE path contained by policy and remediated by patching	Low/Moderate

6.3.8 Discussion

This scenario makes the role of AppArmor explicit. The AppArmor profile is not presented as a replacement for patching `runc`; the fixed `v1.1.12` runtime remains the correct remediation for CVE-2024-21626. Its value is in containment. In the baseline, vulnerable `runc` turns the crafted working directory into a host marker write. In the hardened run, the vulnerable code path is still reachable, but the AppArmor policy denies the filesystem operations that would make the escape useful.

The result supports the layered hardening model used in this chapter. Runtime patching removes known vulnerable behavior, Docker configuration reduces ambient privilege, seccomp narrows kernel entry points, and AppArmor converts workload assumptions into enforceable path and operation rules. When these layers are combined, a container sandbox escape attempt becomes observable and constrained instead of immediately becoming a host integrity compromise.

6.4 Scenario 2: ABRT Privilege Escalation via CVE-2025-12744 Contained by SELinux

6.4.1 Scenario Description

This scenario analyzes CVE-2025-12744 inside a Linux guest VM. The vulnerable component is ABRT, the Automatic Bug Reporting Tool, which is commonly present on Fedora and RHEL-family systems. In the GitLab companion repository’s ABRT scenario, the

VM runs Fedora Linux 42 Server Edition with kernel `6.14.0-63.fc42.x86_64`, package `abrt-2.17.6-4.fc42.x86_64` and SELinux enabled [16], [66].

The vulnerability is a local command injection in ABRT's handling of user-controlled mount or container metadata. The vulnerable code path copies a short attacker-controlled value into a shell command of the form `docker inspect %s`. Because the ABRT event-handling path runs with root privileges, a local unprivileged user can abuse the injection point to execute commands as root inside the VM. The scenario therefore models a guest compromise, not a direct Proxmox-node compromise: the attacker gains administrative control of the VM first and can then steal secrets, alter services or prepare later movement toward Docker or Proxmox assets.

SELinux is relevant because the companion scenario compares the vulnerable default behavior with a hardened configuration where the ABRT event handling boolean is disabled. This does not patch ABRT's input-validation flaw. Instead, it removes the SELinux policy permission that allows the vulnerable ABRT event flow to execute, turning the exploit attempt into a blocked or ineffective local privilege escalation path. The correct remediation remains package update or service removal, while SELinux provides a compensating control during the exposure window.

6.4.2 Threat Model and Preconditions

- **Attacker capability:** the attacker already has a non-root shell inside the VM, either through SSH, a compromised application account or another local foothold. No `sudo` privileges are assumed.
- **Required access:** local command execution on a vulnerable Fedora or RHEL-family guest where ABRT is installed and the ABRT services are running.
- **Target components:** the ABRT daemons `abrt-d`, `abrt-journal-core` and `abrt-oops`, the local ABRT socket, the vulnerable container-data handler, the constructed `docker inspect %s` command and the SELinux boolean `abrt_handle_event`.
- **Assumptions:** the attacker does not initially control the Proxmox node, Docker daemon configuration or any privileged account inside the guest.
- **Assets at risk:** root authority inside the VM, local credentials, application data, logs, deployment secrets, Docker registry material, SSH keys, and any Proxmox API tokens stored in the guest.

6.4.3 Practical Implementation

The companion material records the Fedora, ABRT and SELinux state described above, then uses the referenced public exploit repository to illustrate the privilege transition. This is a reproduction step, not a production recommendation.

```
1 # Unprivileged VM user.
2 id
3 git clone https://github.com/initstring/abrt_root.git
4 cd abrt_root
5 python3 abrt_root.py
6
7 # SELinux hardening control.
8 sudo setsebool -P abrt_handle_event off
9 getsebool abrt_handle_event
10
11 # Vendor remediation path.
12 sudo dnf upgrade --advisory FEDORA-2025-64091db7e0
13
14 # Service reduction if ABRT is not required.
15 sudo systemctl stop abrt-d abrt-journal-core abrt-oops
16 sudo systemctl disable abrt-d abrt-journal-core abrt-oops
```

In the vulnerable baseline, the expected result is a root context inside the VM. With `abrt_handle_event` disabled, the exploit path should no longer produce a root shell, and the attempt should be visible through ABRT, journal, or audit records. The ABRT package must still be upgraded to a fixed release, and ABRT should be disabled entirely on guest systems where automatic crash reporting is not operationally required.

6.4.4 Expected Impact

Without the patch or the SELinux hardening control, CVE-2025-12744 converts a low-privileged local account into root inside the guest VM. From there, the attacker can read protected files, alter services, change logs, install persistence, tamper with ABRT evidence and access credentials that were intended to be available only to administrators or root-owned services.

The immediate impact remains scoped to the VM boundary, but the operational impact can extend further. A VM often stores SSH keys, registry tokens, backup credentials, application secrets, or Proxmox API material. Root access inside the guest can therefore become a preparation step for Docker daemon abuse, lateral movement to other internal systems, or authenticated interaction with the Proxmox management plane.

With `abrt_handle_event` disabled, the vulnerable ABRT package may still be

present, but the event path used by the exploit is not allowed to complete. The residual impact is reduced from root compromise to a failed or blocked escalation attempt that can be investigated. With the vendor patch installed, the vulnerable input handling is removed from the package itself.

6.4.5 Detection Indicators

- **ABRT service activity:** unexpected events in `abrttd`, `abrt-journal-core`, or `abrt-oops`, especially when they originate from ordinary user sessions.
- **Socket and command indicators:** suspicious interaction with `/var/run/abrt/abrt.socket`, abnormal crash-report metadata, or ABRT-spawned command lines involving `docker inspect`.
- **Privilege transition evidence:** an unprivileged user suddenly obtaining `uid=0(root)`, unexpected root shells, modified `/etc/sudoers`, new root-owned helper files, or unauthorized changes to service units and cron entries.
- **SELinux and audit logs:** AVC denials involving ABRT domains, the `abrt_handle_event` policy path, or blocked event processing after the hardening boolean is disabled.
- **Package inventory:** Fedora or RHEL-family guests running vulnerable ABRT versions, such as `abrt-2.17.6-4.fc42.x86_64` in the companion baseline, should be treated as exposed until patched or otherwise mitigated.

6.4.6 Countermeasures

- **Patch ABRT:** upgrade to a fixed ABRT package through the distribution advisory or normal security update channel. SELinux containment is a compensating control, not the vulnerability fix.
- **Disable vulnerable event handling:** set `abrt_handle_event` to `off` with `setsebool -P` when ABRT event handling is not required. This keeps SELinux enforcing while removing the vulnerable policy path used by the exploit.
- **Remove unnecessary ABRT services:** stop and disable `abrttd`, `abrt-journal-core`, and `abrt-oops` on production VMs where automatic crash reporting is not needed.

- **Preserve SELinux enforcement:** do not switch SELinux to disabled mode to work around ABRT or application issues. Use booleans, labels, policy review, and audit diagnostics instead [15], [48], [52].
- **Monitor privilege-sensitive files:** alert on unexpected writes to `/etc/sudoers`, `/etc/cron.d/`, systemd unit paths, root-owned binaries, and SSH key locations.
- **Segment credentials:** avoid storing Proxmox, registry, backup, or SSH secrets in VM locations that become immediately useful after local root compromise.

6.4.7 Risk Analysis

Table 6.2: Qualitative risk assessment for Scenario 2.

Factor	Before countermeasures	Initial value	After countermeasures	Residual value
Likelihood	Vulnerable ABRT package, ABRT services running, event handling enabled, local non-root access	Medium/High	ABRT update, SELinux event-handling boolean disabled, unnecessary ABRT services removed	Low
Impact	Root inside the VM through ABRT command injection	High	Exploit path blocked or removed; evidence available through ABRT, journal, and audit logs	Low/Moderate
Overall risk	Local foothold becomes VM administrator compromise and pivot opportunity	High	Vulnerability patched or contained by SELinux and service reduction	Low/Moderate

6.4.8 Discussion

This scenario narrows the generic idea of VM privilege escalation to a specific SELinux-relevant case. The attacker does not need a misconfigured `sudoers` file or a risky SUID binary. A vulnerable root-running service is enough, provided the attacker can reach the ABRT event path from a local account. In a Proxmox-based infrastructure, that is a serious intermediate compromise: the Proxmox node is still outside the VM boundary, but the attacker now controls the guest and can search for credentials, application secrets, Docker access, and lateral movement opportunities.

The SELinux result should be interpreted as containment, not as a substitute for package maintenance. Disabling `abrt_handle_event` reduces the vulnerable ABRT behavior exposed to local users, and SELinux audit records make failed attempts easier to

investigate. The durable fixes are to install the vendor update or remove ABRT services where they are not required. Combined with credential segmentation and monitoring, these controls reduce a local VM foothold from a likely root compromise to a detectable and much narrower exposure.

6.5 Scenario 3: Redis Lua Sandbox Escape Contained by AppArmor

6.5.1 Scenario Description

This scenario analyzes CVE-2022-0543 as an application sandbox escape inside a Docker workload. The affected Debian and Ubuntu Redis packages exposed the Lua `package` global to Redis scripts. A client able to execute `EVAL` could therefore load Lua `io` or `os` functionality and obtain command execution as the Redis process user [67].

The important distinction is that this vulnerability escapes the Redis Lua sandbox, not the Docker container boundary by itself. The scenario therefore does not model CVE-2022-0543 as a direct host escape. Instead, it evaluates whether AppArmor hardening can contain the post-exploitation phase after the Redis process has already been compromised. In the unconfined baseline, the exploit primitive is able to execute helper commands and create a marker file in the container filesystem. In the hardened run, the same vulnerable Redis package remains present, but the AppArmor profile prevents the sandbox escape from becoming a broader shell-execution and arbitrary file-write primitive.

6.5.2 Threat Model and Preconditions

- **Attacker capability:** the attacker can reach the Redis service and issue Redis commands, including `EVAL`. This may occur because Redis is exposed to an untrusted network segment, protected by weak authentication, or compromised through an application that can submit Redis commands.
- **Required access:** no host-level privilege is assumed. The attacker starts from Redis-level command execution and attempts to turn the Lua sandbox escape into operating system command execution inside the container.
- **Target components:** the vulnerable Redis package `redis-server=5:5.0.7-2`, the embedded Lua runtime, the Docker container process, and the AppArmor profile attached through `--security-opt apparmor=redis-cve-2022-0543`.

- **Assumptions:** both scenario containers use the same Docker runtime restrictions: no added Linux capabilities, `no-new-privileges`, loopback-only port publishing, no privileged mode, and no host bind mounts. The only intentional difference is AppArmor confinement.
- **Assets at risk:** Redis data, secrets available to the Redis process, writable container paths, application integrity, and any downstream systems reachable from the compromised workload.

6.5.3 Practical Implementation

The GitLab companion implementation builds a vulnerable Redis image from `ubuntu:20.04`, pins the original Focal package version `redis-server=5:5.0.7-2`, and compares two containers. The baseline container is started with `apparmor=unconfined` on `127.0.0.1:63791`. The hardened container is started on `127.0.0.1:63792` with the custom AppArmor profile `redis-cve-2022-0543`.

```

1  # Repository comparison from the GitLab companion repository.
2  sudo -v
3  ./run-demo.sh compare
4
5  # Client probes used by the script.
6  python3 scripts/redis_lab_client.py --port 63791 package
7  python3 scripts/redis_lab_client.py --port 63791 eval-id
8  python3 scripts/redis_lab_client.py --port 63791 eval-touch
9
10 python3 scripts/redis_lab_client.py --port 63792 package
11 python3 scripts/redis_lab_client.py --port 63792 eval-id
12 python3 scripts/redis_lab_client.py --port 63792 eval-touch

```

The `package` probe reports whether the vulnerable Lua global is exposed in both containers. The `eval-id` probe uses the Lua sandbox escape to attempt `io.popen("id")`. The `eval-touch` probe attempts to create `/tmp/redis-cve-2022-0543-poc`. In the baseline run, these probes demonstrate process-level command execution as the Redis user and a file write inside the container filesystem. In the hardened run, Redis remains functional for legitimate `PING` and `SET/GET` operations, but the exploit follow-up is blocked or produces no useful command output.

The custom profile is deliberately workload-specific. It permits Redis to read its configuration, use `/data`, listen on TCP, and map the Lua library required by the vulnerable package. It denies shell and helper execution, mount and ptrace operations, selected dangerous capabilities, writes to system paths, and writes to temporary locations such as

`/tmp/**`. AppArmor is therefore used as a containment layer after the CVE has been triggered: the vulnerable code path is still reachable, but the operating-system actions available to the compromised Redis process are reduced by kernel policy [12], [13], [50].

6.5.4 Expected Impact

Without AppArmor confinement, successful exploitation gives the attacker command execution in the Redis container as the Redis user. The attacker may inspect process identity, write marker files, tamper with data that Redis can access, stage additional tools, or attempt further pivots using any reachable files, binaries, sockets, or network destinations. The impact remains below direct host compromise when Docker is not privileged and no host socket or bind mount is exposed, but it is still a serious workload compromise.

With the custom AppArmor profile enforced, the residual impact is narrower. The Redis vulnerability remains present and the Lua sandbox escape is still attempted, but shell execution and uncontrolled file writes are denied. This does not replace patching Redis; it reduces the damage window while a vulnerable workload exists or while patching is being completed.

6.5.5 Detection Indicators

- **Redis activity:** unexpected EVAL commands, Lua scripts that reference `package.loadlib`, `io.popen`, or `os.execute`, and unusual command patterns from application clients.
- **Container filesystem changes:** creation of unexpected files such as `/tmp/redis-cve-2022-0543-poc`, modified Redis data files, or new tooling inside writable paths.
- **AppArmor denials:** kernel records containing `apparmor="DENIED"` for profile `redis-cve-2022-0543`, especially denied execution of `/bin/sh`, `/usr/bin/id`, `/usr/bin/touch`, or writes under `/tmp/**`.
- **Runtime telemetry:** unexpected child-process attempts from `redis-server`, blocked mount or ptrace operations, and abnormal outbound traffic from the Redis container.

6.5.6 Countermeasures

- **Patch the vulnerable package:** update Redis to a fixed Debian or Ubuntu package version. AppArmor containment is a compensating control, not the vulnerabil-

ity fix.

- **Constrain Redis exposure:** bind Redis to trusted interfaces, require authentication where appropriate, and prevent untrusted clients from issuing arbitrary **EVAL** commands.
- **Apply workload-specific AppArmor:** attach a profile that permits only Redis' expected configuration, data, library, and socket behavior while denying generic shell execution, helper binaries, temporary-file writes, ptrace, mount operations, and unnecessary capabilities.
- **Preserve Docker isolation:** avoid `--privileged`, avoid host bind mounts and `/var/run/docker.sock`, drop all unnecessary capabilities, enforce `no-new-privileges`, and publish service ports only where needed [2], [4], [37].
- **Monitor AppArmor and Redis logs:** forward kernel denials, Redis command logs, and Docker events to centralized monitoring so attempted sandbox escapes are visible even when blocked.

6.5.7 Risk Analysis

Table 6.3: Qualitative risk assessment for Scenario 3.

Factor	Before countermeasures	Initial value	After countermeasures	Residual value
Likelihood	Vulnerable Redis package reachable through EVAL	High	Patched package, restricted exposure, AppArmor profile	Low/Moderate
Impact	Command execution as the Redis process inside the container	High	Shell execution and uncontrolled writes denied; Redis service remains usable	Moderate
Overall risk	Application sandbox escape becomes broad workload compromise	High	CVE impact contained by runtime policy and monitoring	Low/Moderate

6.5.8 Discussion

This scenario shows the specific value of AppArmor hardening in a containerized workload: it does not make Redis invulnerable and it does not transform CVE-2022-0543 into a host escape. Its value is in limiting what the compromised Redis process can do after the application-level sandbox has failed. The same exploit probes that succeed against the

unconfined baseline are reduced to blocked or empty results when the profile is attached, while ordinary Redis functionality continues to work.

The result supports the defense-in-depth model used throughout this chapter. Patching removes the vulnerability, Docker runtime settings reduce the container’s ambient privilege, and AppArmor turns workload assumptions into enforceable kernel rules. When these layers are combined, an application sandbox escape remains a security incident, but it is less likely to expand into persistence, tool staging, or host-facing compromise.

6.6 Scenario 4: Proxmox VE API Trust-Boundary Abuse via CVE-2024-21545

6.6.1 Scenario Description

The Proxmox VE REST API is the management plane for VMs, containers, storage, and cluster configuration. This scenario narrows the generic management-plane risk to CVE-2024-21545, a trust-boundary issue involving Proxmox API handling of QEMU Guest Agent responses. In the GitLab companion scenario, a compromised guest VM runs a modified QEMU Guest Agent that returns a crafted `download` object. On a vulnerable Proxmox VE host, an authorized API request to the guest-agent endpoint can then cause the host to read a local file, such as `/etc/hostname`, and return its contents through the API response [16], [68]–[70].

The scenario is intentionally narrower than full API compromise. The attacker is assumed to have root inside the guest VM and API permissions for guest-agent commands on the target VM, not unrestricted cluster-administrator access. The companion workflow does not modify host files, install persistence, enumerate the cluster or attempt VM takeover. Its security meaning is still significant: an API path that should mediate guest operations becomes a host-file-read primitive when a vulnerable Proxmox service trusts attacker-controlled guest-agent output.

6.6.2 Threat Model and Preconditions

- **Attacker capability:** the attacker already controls the guest VM sufficiently to replace or modify `qemu-ga`, and can reach the Proxmox API with credentials or a token authorized to issue guest-agent commands for that VM.
- **Required access:** root access inside the guest VM, QEMU Guest Agent enabled, a vulnerable Proxmox VE node, and an API account with the minimum permissions

needed for guest-agent operations, such as `VM.Monitor` on the target VM.

- **Target components:** `qemu-guest-agent` inside the VM, the Proxmox API endpoint for guest-agent execution, `pveproxy`, the host-side API response handling path, and local files readable by the vulnerable host process.
- **Assumptions:** the API account is scoped to the target VM, the first target file is non-sensitive, and the scenario is run on an isolated vulnerable node.
- **Assets at risk:** Proxmox host file confidentiality, management-plane trust boundaries, API audit integrity, VM administration permissions and any sensitive host files reachable through the vulnerable response path.

6.6.3 Practical Implementation

The GitLab companion repository contains the supporting implementation for this scenario. It uses a modified QEMU Guest Agent source file, `main.c`, to rewrite an error response containing a command name such as `NOTFOUND/etc/hostname` into a JSON object of the following form:

```
1 {"return":{"download":{"path":"/etc/hostname","content-type":"text/plain\n  "}}}
```

The patched guest agent is compiled from QEMU source and installed only inside the target VM. The following commands summarize the GitLab companion workflow.

```
1 # Build host: replace qga/main.c with the modified source
2 # published in the GitLab companion repository.
3 git clone https://gitlab.com/qemu-project/qemu.git
4 cd qemu
5 git checkout v10.2.1
6 ./configure --enable-guest-agent --enable-debug
7 make -j"$(nproc)" qemu-ga
8
9 # Guest VM: install the patched guest agent.
10 scp build/qga/qemu-ga root@<vm-ip>:/tmp/qemu-ga.patched
11 ssh root@<vm-ip>
12 cp /usr/sbin/qemu-ga /usr/sbin/qemu-ga.original
13 install -m 0755 /tmp/qemu-ga.patched /usr/sbin/qemu-ga
14 systemctl restart qemu-guest-agent
```

The API trigger then authenticates to Proxmox and sends a guest-agent command that fails inside the VM while embedding the requested host path after the `NOTFOUND` marker:

```

1 # Obtain a ticket and CSRF token.
2 curl -k \
3   -d "username=<user>@<realm>&password=<password>" \
4     "https://<proxmox-host>:8006/api2/json/access/ticket"
5
6 # Trigger the guest-agent path with a non-sensitive host file first.
7 curl -k \
8   -b "PVEAuthCookie=<ticket>" \
9   -H "CSRFPreventionToken: <csrf-token>" \
10  -X POST \
11  "https://<proxmox-host>:8006/api2/json/nodes/<node>/qemu/<vmid>/agent/
    exec" \
12  -d "command=NOTFOUND/etc/hostname"

```

On a vulnerable host, the response contains the contents of `/etc/hostname` from the Proxmox node rather than from the guest VM. The patched guest-agent log in the VM should also show the debug line `[CVE-2024-21545] Reading file: /etc/hostname`. On a fixed host, the API should not return the host file. If the patched guest agent is not running, the request behaves like a normal failed guest-exec call.

6.6.4 Expected Impact

Without the Proxmox fix, the attacker can turn a compromised guest agent and an authorized guest-agent API call into a host-file-read primitive. The repository workflow reads a harmless host file, but the same trust-boundary failure may expose more sensitive host-local files depending on service permissions and file readability. This does not by itself prove full cluster compromise, VM deletion or arbitrary host write capability; it is a confidentiality failure across the guest-host boundary.

The impact becomes more serious when combined with overprivileged API accounts, exposed management networks or sensitive material readable by the affected Proxmox service. In a production cluster, host file disclosure can support credential theft, reconnaissance, or follow-on attacks against the management plane.

6.6.5 Detection Indicators

- **API access logs:** unexpected calls to guest-agent endpoints such as `/nodes/<node>/qemu/<vmid>/agent/exec`, especially when command parameters include unusual strings or path-like values.
- **Guest-agent logs:** messages from the modified or suspicious guest agent, unexpected guest-agent failures, or debug strings such as `[CVE-2024-21545] Reading`

file:

- **Authentication and authorization traces:** use of API accounts or tokens with `VM.Monitor` or broader permissions outside expected automation windows.
- **Host response anomalies:** API responses containing file content or `download` objects where a normal guest-exec failure should have been returned.
- **Guest integrity signals:** modified `/usr/sbin/qemu-ga`, unexpected service restarts, package verification failures or guest-agent binaries not matching the distribution package.

6.6.6 Countermeasures

- **Patch Proxmox VE:** install the vendor security update that fixes CVE-2024-21545. MAC policy and API scoping reduce exposure, but they are not substitutes for the Proxmox fix.
- **Scope guest-agent permissions:** grant guest-agent API permissions only to accounts that require them, only for the relevant VMs, and preferably through named tokens rather than shared administrator credentials.
- **Restrict management access:** expose port 8006 only to a management subnet or VPN and prevent ordinary workloads from making arbitrary outbound connections to the Proxmox API.
- **Harden guest integrity:** monitor `qemu-ga` package integrity, service restarts and unexpected binary replacement inside guests where the agent is enabled.
- **Audit API usage:** alert on unusual guest-agent commands, repeated guest-exec failures, unexpected `download` responses, off-hours API activity and token use from unfamiliar sources.
- **Apply local confinement where practical:** use AppArmor or SELinux policy to reduce the host files that Proxmox services can read, recognizing that such confinement is a compensating control and must be evaluated carefully against Proxmox functionality.

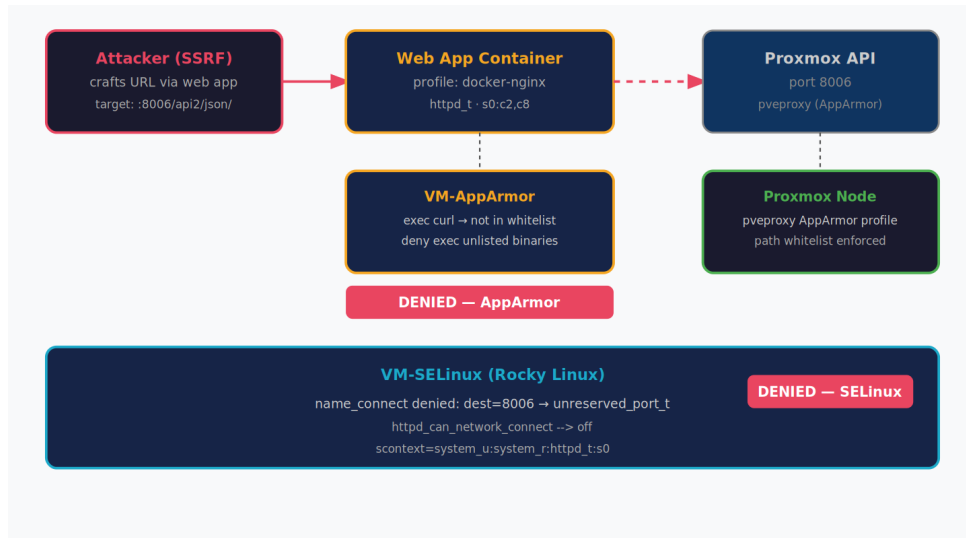


Figure 6.2: Guest-agent and Proxmox API trust-boundary abuse constrained by patching, API scoping, network policy, and monitoring.

6.6.7 Risk Analysis

Table 6.4: Qualitative risk assessment for Scenario 4.

Factor	Before counter-measures	Initial value	After countermeasures	Residual value
Likelihood	Vulnerable Proxmox node, modified guest agent, API account authorized for guest-agent commands	Medium	Proxmox update, scoped <code>VM.Monitor</code> , management-network restriction, guest integrity monitoring	Low
Impact	Host file disclosure through an API/guest-agent trust-boundary failure	High	Vulnerable response path removed; blast radius reduced by permission scoping and host confinement	Low/Moderate
Overall risk	Guest compromise can cross into host-file confidentiality impact	High	Vulnerability patched and guest-agent API exposure reduced	Low/Moderate

6.6.8 Discussion

This scenario differs from the runc and Redis cases because the failure is not primarily a container sandbox problem. The attack path crosses a trust boundary between the

guest agent and the Proxmox API. The guest VM is already compromised, and the vulnerable host incorrectly handles attacker-influenced guest-agent output. The appropriate lesson is therefore not that AppArmor or SELinux can replace vendor patching, but that management-plane interfaces and guest integration channels must be treated as privileged attack surfaces.

The scenario also clarifies the limits of the claim. It demonstrates host-file read through a vulnerable API/guest-agent path; it does not demonstrate arbitrary host write, unauthenticated access, cluster takeover or a generic Proxmox API brute-force attack. Those remain relevant operational risks, but they are separate from the CVE-2024-21545 path and should be mitigated through ordinary identity, network and audit controls.

6.7 Comparative Risk Matrix

Table 6.5 summarizes the four detailed scenarios and the principal controls that reduce residual exposure.

Table 6.5: Comparative risk matrix for the analysed attack scenarios.

Scenario	Target asset	Security properties	Likelihood	Impact	Initial risk	Main countermeasures	Residual risk
Runtime escape via CVE-2024-21626	VM running Docker/runc	Host filesystem integrity	Medium/High	High	High	runc update, AppArmor profile, seccomp, no privileged mode, no socket mounts	Low/Moderate
ABRT privilege escalation via CVE-2025-12744	Guest VM	VM confidentiality, integrity and authorization	Medium/High	High	High	ABRT update, SELinux boolean hardening, service reduction, credential segmentation	Low/Moderate
Redis Lua sandbox escape	Redis workload and container filesystem	Workload confidentiality and integrity	High	High	High	Redis patching, restricted exposure, AppArmor profile, no privileged mode	Low/Moderate
Proxmox CVE-2024-21545 API/guest-agent trust-boundary abuse	Proxmox host	Host file confidentiality and management-plane trust	Medium	High	High	Proxmox update, scoped guest-agent permissions, management-network isolation, guest integrity monitoring	Low/Moderate

6.8 General Countermeasure Strategy

The scenario analysis leads to several cross-cutting countermeasures:

- **Defense in depth:** combine VM isolation, container runtime controls, MAC profiles, seccomp, firewall rules, and backups rather than relying on a single mechanism.
- **Security lifecycle integration:** include vulnerability scanning, image review, configuration review, and Infrastructure as Code validation in CI/CD.

- **Strict access control:** apply least privilege to users, API tokens, containers, and services. Administrative accounts should use MFA and should be separated by operational role.
- **Continuous monitoring:** centralize logs from Proxmox, Docker, the kernel, AppArmor, SELinux, SSH, and network devices. Detection rules should cover privileged container launches, host mounts, API abuse, and lateral movement [65].
- **Response and recovery:** define playbooks for isolating compromised VMs, rotating keys, revoking API tokens, preserving evidence, and restoring trusted backups.
- **Encryption and backup:** encrypt sensitive data at rest where feasible and rehearse backup restoration regularly.
- **Awareness and training:** document approved container patterns, prohibited mount and privilege settings, and the operational meaning of AppArmor and SELinux denials.

These measures compose a multilayer security posture that combines preventive controls, detection, and recovery. This is consistent with container hardening guidance from NIST and with Kubernetes hardening recommendations from NSA and CISA, especially where privileged containers, host namespaces, and host filesystem exposure are concerned [3]–[5].

6.9 Chapter Summary

This chapter has translated the paper’s security model into four concrete scenarios for a Proxmox-based container infrastructure. The runc scenario shows how a vulnerable container runtime can turn a crafted working directory into a host write unless the runtime is patched or the process is confined. The ABRT scenario shows how a local guest foothold can become root inside the VM and why SELinux policy controls are useful compensating measures. The Redis scenario shows that an application sandbox escape is not automatically a host escape, but still requires workload-specific confinement. The Proxmox CVE-2024-21545 scenario shows that guest integration and management-plane APIs must be treated as privileged trust boundaries.

The analysis assumes a Debian-based Proxmox node, an AppArmor-enabled Ubuntu VM for the AppArmor and Docker scenarios, a Fedora/RHEL-family guest for the SELinux ABRT scenario, an isolated internal network, and no additional endpoint detection unless explicitly discussed. These assumptions are intentionally conservative: they

make clear which layer is expected to enforce each boundary and which residual risks remain after hardening.

Overall, the chapter supports the central claim of the paper: containerized and virtualized environments introduce meaningful attack surfaces, but disciplined application of patching, least privilege, MAC enforcement, network segmentation, monitoring, and configuration review can substantially reduce the likelihood and impact of the scenarios analysed here.

Chapter 7

Conclusions

This paper has argued that hardening Docker workloads on Proxmox VE is most effective when isolation, privilege restriction, system-call filtering, and Mandatory Access Control enforcement are treated as complementary layers rather than as interchangeable controls. Namespaces, cgroups, capabilities, and seccomp establish the baseline container containment model by partitioning selected kernel resources, constraining resource consumption, reducing ambient privilege, and narrowing access to kernel entry points. Their value is substantial, but bounded: they reduce attack surface and blast radius, while still depending on correct kernel behavior, secure runtime configuration, and sound application design [2]–[4], [8]–[10].

The discussion of AppArmor and SELinux extends that baseline with mandatory policy enforcement. AppArmor’s path-based profiles and SELinux’s label- and type-based confinement both allow administrators to express restrictions that remain enforceable even after a process is compromised. Their operational trade-off is not simply one of strength versus weakness. AppArmor is often direct to author and reason about for specific programs and filesystem paths, whereas SELinux provides a richer label-based model that can be more robust across complex systems when labeling and policy management are correct. The appropriate choice therefore depends on the host distribution, administrative expertise, workload shape, and the kind of policy boundary that must be maintained [11]–[15].

The scenario analysis in Chapter 6 reinforces the same conclusion in operational terms. In the runc CVE-2024-21626 case, the unconfined vulnerable runtime creates a host marker, the AppArmor-constrained run blocks the same write path, and the fixed runtime removes the vulnerable behavior. The ABRT CVE-2025-12744 scenario shows that guest hardening still matters even when the container layer is restricted, because root inside a VM can expose credentials, persistence paths, and lateral movement opportunities. The

Redis CVE-2022-0543 scenario shows that an application sandbox escape remains a workload compromise, but workload-specific AppArmor policy can prevent it from expanding into generic shell execution and uncontrolled file writes. The Proxmox VE CVE-2024-21545 scenario shows a different class of risk: an authorized API path and a compromised guest agent can cross a host-file trust boundary on vulnerable Proxmox nodes, so vendor patching, guest-agent permission scoping, management-network isolation, API logging, and least-privilege tokens become decisive controls [5], [65], [68], [69].

The comparative risk matrix in Table 6.5 therefore supports the central claim of the paper. Patching, least privilege, seccomp, AppArmor or SELinux enforcement, network segmentation, monitoring, and configuration review can substantially reduce both likelihood and impact across runtime escape, guest escalation, application sandbox escape, and management-plane trust-boundary scenarios. At the same time, the controls do not provide a mathematical proof of safety. Misconfiguration, vulnerable application logic, exposed credentials, supply-chain compromise, and kernel-level defects remain residual risks that must be managed through secure development, image provenance, timely patching, credential hygiene, logging, monitoring, and recovery planning [1]–[4], [37].

The main limitation of the evaluation is that it is qualitative. The scenario chapter uses repository-backed implementations, enforce/deny outcomes, detection indicators, and comparative residual-risk judgments; it does not claim measured exploit probabilities, performance overheads, or statistical confidence. This is appropriate for showing how the layers interact, but it leaves open questions about policy cost, operational compatibility, and performance impact under realistic production workloads [16].

Future work should therefore extend the qualitative analysis into a more rigorous empirical campaign. Useful next steps include measuring AppArmor and SELinux overhead under representative Docker workloads, comparing policy maintenance cost across Ubuntu, Fedora and Rocky Linux deployments, extending the scenario set to cover VM escape and security misconfiguration in greater depth, and integrating policy review into CI/CD and Infrastructure as Code pipelines. In that model, patching, image review, runtime hardening, MAC policy enforcement, and configuration review become part of the operational lifecycle rather than post-deployment remediation.

Bibliography

- [1] *Docker engine security*, Accessed 2026. [Online]. Available: <https://docs.docker.com/engine/security/>.
- [2] Docker Documentation Authors, *Seccomp security profiles for Docker*, Accessed: Jun. 3, 2026. [Online]. Available: <https://docs.docker.com/engine/security/seccomp/>.
- [3] Kubernetes Documentation Authors, *Linux kernel security constraints for Pods and containers*, Accessed: Jun. 3, 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/security/linux-kernel-security-constraints/>.
- [4] M. Souppaya, J. Morello, and K. Scarfone, “Application container security guide,” National Institute of Standards and Technology, NIST Special Publication 800-190, Sep. 2017. DOI: 10.6028/NIST.SP.800-190. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-190>.
- [5] National Security Agency and Cybersecurity and Infrastructure Security Agency, “Kubernetes Hardening Guidance,” National Security Agency and Cybersecurity and Infrastructure Security Agency, Tech. Rep., Aug. 2022, Version 1.2. Accessed: 2026-06-05. [Online]. Available: https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2_20220829.PDF.
- [6] P. Krzyzanowski, *Containment*, CS 419 course notes, Rutgers University. Accessed: 2026-03-10, Mar. 2025. [Online]. Available: <https://pk.org/classes/419/notes/containment.html>.
- [7] IBM. “Containers versus virtual machines (vms): What’s the difference?” Accessed: 2026-03-10. (2024), [Online]. Available: <https://www.ibm.com/think/topics/containers-vs-vms>.
- [8] M. Kerrisk, *Namespaces(7) — linux manual page*, Linux man-pages project, 2024. [Online]. Available: <https://man7.org/linux/man-pages/man7/namespaces.7.html>.

- [9] M. Kerrisk, *Cgroups(7) - linux manual page*, Linux man-pages project, 2024. [Online]. Available: <https://man7.org/linux/man-pages/man7/cgroups.7.html>.
- [10] M. Kerrisk, *Capabilities(7) — linux manual page*, Linux man-pages project. [Online]. Available: <https://man7.org/linux/man-pages/man7/capabilities.7.html>.
- [11] Linux Kernel Documentation. “Linux security module development.” Accessed: 2026-06-26. (2026), [Online]. Available: <https://docs.kernel.org/security/lsm-development.html>.
- [12] Linux Kernel Documentation. “Apparmor.” Accessed: 2026-05-29. (2026), [Online]. Available: <https://docs.kernel.org/admin-guide/LSM/apparmor.html>.
- [13] AppArmor Project, *Apparmor.d(5) – syntax of security profiles for apparmor*, Ubuntu Manpages. Accessed: 2026-05-29, 2026. [Online]. Available: <https://manpages.ubuntu.com/manpages/questing/man5/apparmor.d.5.html>.
- [14] P. Loscocco and S. Smalley, “Integrating flexible support for security policies into the linux operating system,” in *Proceedings of the FREENIX Track: 2001 USENIX Annual Technical Conference*, USENIX Association, 2001. [Online]. Available: https://www.usenix.org/legacy/events/usenix01/freenix01/full_papers/loscocco/loscocco.pdf.
- [15] Red Hat Documentation. “Using selinux.” Red Hat Enterprise Linux 9 documentation. Accessed: 2026-06-26. (2026), [Online]. Available: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/using_selinux/index.
- [16] “CNS Code Examples Companion Repository.” Public GitLab companion repository. Accessed: 2026-06-29. (2026), [Online]. Available: <https://gitlab.com/giancarlopadovan/cns-codeexamples>.
- [17] J. E. Smith and R. Nair, “Virtualization: A survey of virtualization technologies,” *IEEE Computer*, vol. 38, no. 5, pp. 32–38, 2005.
- [18] S. Campbell and M. Jeronimo, “An introduction to virtualization,” *Published in “Applied Virtualization”, Intel*, pp. 1–15, 2006.
- [19] IBM. “What is virtualization?” Accessed: 10 March 2026. (2025), [Online]. Available: <https://www.ibm.com/think/topics/virtualization>.
- [20] “Qemu/kvm virtual machines.” Accessed: 11 March 2026. (2026), [Online]. Available: https://pve.proxmox.com/wiki/QEMU/KVM_Virtual_Machines.
- [21] A. M. Potdar, N. D. G. S. Kengond, and M. M. Mulla, “Performance evaluation of docker container and virtual machine,” *Procedia Computer Science*, vol. 171, pp. 1419–1428, 2020. DOI: 10.1016/j.procs.2020.04.152.

- [22] S. Nimmagadda, “Linux namespaces and cgroups as os primitives for lightweight virtualization: Architecture, isolation mechanisms, and performance evaluation,” *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, vol. 9, no. 2, pp. 811–822, 2018. DOI: 10.61841/turcomat.v9i2.15258. [Online]. Available: <https://doi.org/10.61841/turcomat.v9i2.15258>.
- [23] CubePath Support Team. “Linux namespaces and cgroups: Container technology foundations guide.” Accessed: 2026-03-10. (Jan. 2026), [Online]. Available: <https://cubepath.com/docs/advanced-topics/linux-namespaces-and-cgroups>.
- [24] Google Cloud. “Containers vs. virtual machines (vms).” Accessed: 2026-03-10. (2024), [Online]. Available: <https://cloud.google.com/discover/containers-vs-vms>.
- [25] OVHcloud. “Container vs virtual machine (vm).” Accessed: 2026-03-10. (2024), [Online]. Available: <https://www.ovhcloud.com/asia/learn/container-vs-vm/>.
- [26] C. Pahl, “Containerization and the paas cloud,” *Procedia Computer Science*, vol. 170, pp. 377–384, 2020. DOI: 10.1016/j.procs.2020.03.070. [Online]. Available: <https://doi.org/10.1016/j.procs.2020.03.070>.
- [27] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, “An updated performance comparison of virtual machines and linux containers,” in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, IEEE, 2015, pp. 171–172. DOI: 10.1109/ISPASS.2015.7095802.
- [28] S. J. Bigelow. “Containers vs. vms: What are the key differences?” TechTarget. (Mar. 2024), [Online]. Available: <https://www.techtarget.com/searchitoperations/tip/Containers-vs-VMs-What-are-the-key-differences> (visited on 03/12/2026).
- [29] D. Merkel, “Docker: Lightweight linux containers for consistent development and deployment,” *Linux Journal*, no. 239, 2014.
- [30] C. Pahl, “Containerization and the paas cloud,” *IEEE Cloud Computing*, vol. 2, no. 3, pp. 24–31, 2015. DOI: 10.1109/MCC.2015.51.
- [31] Linux Kernel Documentation. “Control group v2.” Accessed: 2026-05-29. (2026), [Online]. Available: <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>.
- [32] Cilium Authors, *BPF and XDP Reference Guide*, <https://docs.cilium.io/en/latest/reference-guides/bpf/index.html>, Cilium 1.20.0-dev documentation. Last updated: June 3, 2026. Accessed: June 3, 2026, 2026.

- [33] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975. DOI: 10.1109/PROC.1975.9939.
- [34] libcap project, *Setcap(8) - linux manual page*, Linux man-pages project, 2024. [Online]. Available: <https://man7.org/linux/man-pages/man8/setcap.8.html>.
- [35] libcap project, *Cap_text_formats(7) - linux manual page*, Linux man-pages project, 2024. [Online]. Available: https://man7.org/linux/man-pages/man7/cap_text_formats.7.html.
- [36] M. Kerrisk, *User_namespaces(7) - linux manual page*, Linux man-pages project, 2024. [Online]. Available: https://man7.org/linux/man-pages/man7/user_namespaces.7.html.
- [37] Docker Documentation. “Runtime privilege and linux capabilities.” Accessed: 2026-05-29. (2026), [Online]. Available: <https://docs.docker.com/engine/containers/run/#runtime-privilege-and-linux-capabilities>.
- [38] Kubernetes Documentation Authors. “Pod security standards.” Accessed: 2026-03-13. (2026), [Online]. Available: <https://kubernetes.io/docs/concepts/security/pod-security-standards/>.
- [39] S. Lanchec. “Access control techniques.” Accessed: 2026-03-16. (Jun. 2024), [Online]. Available: <https://www.forestadmin.com/blog/access-control-mechanisms>.
- [40] phoenixNAP. “What is discretionary access control?” Accessed: 2026-03-16. (Aug. 2025), [Online]. Available: <https://phoenixnap.com/glossary/discretionary-access-control>.
- [41] A. Devasia. “What is discretionary access control? full guide.” Accessed: 2026-03-16. (2024), [Online]. Available: <https://getsafeandsound.com/blog/what-is-discretionary-access-control/>.
- [42] Tech Prescient. “Dac vs mac access control.” Accessed: 2026-03-16. (2023), [Online]. Available: <https://www.techprescient.com/blogs/dac-vs-mac-access-control/>.
- [43] Frontegg. “Access control in security: Complete guide.” Accessed: 2026-03-16. (2024), [Online]. Available: <https://frontegg.com/guides/access-control-in-security>.
- [44] E. Risk. “Access control models: Mac, dac, rbac, & pam explained.” Accessed: 2026-03-13. (Jul. 2021), [Online]. Available: <https://www.twingate.com/blog/other/access-control-models>.

- [45] TrustCloud. “Designing an effective access control policy: Best practices and key considerations.” Accessed: 2026-03-16. (2025), [Online]. Available: <https://community.trustcloud.ai/docs/grc-launchpad/grc-101/governance/designing-an-effective-access-control-policy-best-practices-and-key-considerations/>.
- [46] Canonical Ltd. “Apparmor.” Ubuntu Security Documentation. Last updated: 2026-03-09. Accessed: 2026-05-29. (2026), [Online]. Available: <https://documentation.ubuntu.com/security/security-features/privilege-restriction/apparmor/>.
- [47] Ubuntu Documentation Team. “Apparmor.” Ubuntu Server Documentation. Accessed: 2026-05-29. (2026), [Online]. Available: <https://ubuntu.com/server/docs/how-to/security/apparmor>.
- [48] Fedora Project Documentation Team, *Selinux troubleshooting*, Fedora Docs, Accessed: 2026-06-03. [Online]. Available: <https://docs.fedoraproject.org/en-US/quick-docs/selinux-troubleshooting/>.
- [49] Kubernetes Documentation Authors, *Configure a Security Context for a Pod or Container*, Accessed: Jun. 3, 2026. [Online]. Available: <https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>.
- [50] Docker Documentation Authors. “Apparmor security profiles for docker.” Accessed: 2026-05-29. (2026), [Online]. Available: <https://docs.docker.com/engine/security/apparmor/>.
- [51] Kubernetes Documentation Authors, *Restrict a Container’s Access to Resources with AppArmor*, Accessed: Jun. 3, 2026. [Online]. Available: <https://kubernetes.io/docs/tutorials/security/apparmor/>.
- [52] Red Hat. “What is selinux?” Published: 30 Aug 2019. Accessed: 2026-03-16. (Aug. 2019), [Online]. Available: <https://www.redhat.com/en/topics/linux/what-is-selinux>.
- [53] M. Maleki, *Ln command to add symbolic and hard links to files or directories*, <https://pressbooks.senecapolytechnic.ca/uli101/chapter/ln-for-adding-links-between-files/>, Chapter: Links and inodes in Linux file system. Accessed: 2026-05-29, 2022.

- [54] SUSE LLC. “Introducing apparmor.” Security and Hardening Guide, SUSE Linux Enterprise Server 15 SP7. Accessed: 2026-05-29. (2026), [Online]. Available: <https://documentation.suse.com/sles/15-SP7/html/SLES-all/cha-apparmor-intro.html>.
- [55] Docker Documentation Authors, *docker container run*, Accessed: Jun. 3, 2026. [Online]. Available: <https://docs.docker.com/reference/cli/docker/container/run/>.
- [56] Docker Documentation Authors, *Bind mounts*, Accessed: Jun. 3, 2026. [Online]. Available: <https://docs.docker.com/engine/storage/bind-mounts/>.
- [57] C. Meadows, S. Hounsinnou, T. Wood, and G. Bloom, “Sidecar-based path-aware security for microservices,” in *Proceedings of the 28th ACM Symposium on Access Control Models and Technologies*, New York, NY, USA: Association for Computing Machinery, 2023, pp. 157–162, ISBN: 9798400701733. DOI: 10.1145/3589608.3594742. [Online]. Available: <https://doi.org/10.1145/3589608.3594742>.
- [58] Wikipedia contributors. “Security-enhanced linux.” Accessed: 2026-03-16. (2026), [Online]. Available: https://en.wikipedia.org/wiki/Security-Enhanced_Linux.
- [59] GeeksforGeeks. “What is selinux?” Last updated: 13 Nov 2025. Accessed: 2026-03-16. (Nov. 2025), [Online]. Available: <https://www.geeksforgeeks.org/linux-unix/what-is-selinux/>.
- [60] The CyberSec Guru, *The ultimate guide to selinux: From “permission denied” to policy master*, Accessed: 2026-05-29, Oct. 2025. [Online]. Available: <https://thecybersecguru.com/tutorials/selinux-ultimate-guide/>.
- [61] Joint Task Force Transformation Initiative, “Guide for conducting risk assessments,” National Institute of Standards and Technology, NIST Special Publication 800-30 Revision 1, Sep. 2012. DOI: 10.6028/NIST.SP.800-30r1. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-30r1>.
- [62] OWASP Foundation. “OWASP Risk Rating Methodology.” Accessed: 2026-06-05. (2026), [Online]. Available: https://owasp.org/www-community/OWASP_Risk_Rating_Methodology.
- [63] National Vulnerability Database. “CVE-2024-21626 Detail.” Accessed: 2026-06-29. (2024), [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2024-21626>.

- [64] Open Containers Initiative. “runc Security Advisory GHSA-xr7r-f8xq-vfvv.” Accessed: 2026-06-29. (2024), [Online]. Available: <https://github.com/opencontainers/runc/security/advisories/GHSA-xr7r-f8xq-vfvv>.
- [65] MITRE ATT&CK. “Escape to Host, Technique T1611.” Accessed: 2026-06-05. (2026), [Online]. Available: <https://attack.mitre.org/techniques/T1611/>.
- [66] National Vulnerability Database. “CVE-2025-12744 Detail.” Accessed: 2026-06-29. (2025), [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2025-12744>.
- [67] National Vulnerability Database. “CVE-2022-0543 Detail.” Accessed: 2026-06-29. (2022), [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2022-0543>.
- [68] Proxmox Server Solutions GmbH. “Proxmox VE API.” Accessed: 2026-06-05. (2026), [Online]. Available: https://pve.proxmox.com/wiki/Proxmox_VE_API.
- [69] National Vulnerability Database. “CVE-2024-21545 Detail.” Accessed: 2026-06-29. (2024), [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2024-21545>.
- [70] Snyk Labs. “Proxmox VE CVE-2024-21545: Tricking the API.” Accessed: 2026-06-29. (2024), [Online]. Available: <https://labs.snyk.io/resources/proxmox-ve-cve-2024-21545-tricking-the-api/>.